

St. Mary's University



Computer Science Department

Blockchain-Based E-Voting System

A Project Presented to the Department of Computer Science in Partial Fulfilment of the
Requirements of the Degree Bachelor of Science

Done by: Bethlehem Fekadu, Merry Ahadu, Nehemiah Mekonnen, Niya Desta

Advisor: Dagmawi Yewlsew

Addis Ababa, Ethiopia June 06, 2024

Authentication of our work

Advisee's Declaration

We, the undersigned, declare that this senior project paper is our original work. Prepared under the guidance of Dagmawi Yewlsew. All sources of materials used for the manuscript have been duly acknowledged.

Group members

Signature

Bethlehem Fekadu

Merry Ahadu

Nehemiah Mekonnen

Niya Desta

Date of Submission: 06-06-2024

Advisor's Declaration

The paper has been submitted for the examination with my approval as the University advisor.

Name: Dagmawi Yewlsew

Signature: _____

Date: - 06-06-2024

APPROVED BY THE COMMITTEE

CHAIRPERSON

SIGNATURE _____ DATE _____

ADVISOR

SIGNATURE _____ DATE _____

EXAMINER

SIGNATURE _____ DATE _____

Acknowledgment

We extend our heartfelt gratitude to the individuals whose contributions have been pivotal to the success of our project. Foremost among them, we express our most profound appreciation to our esteemed advisor, Dagmawi Yewlsew, whose unwavering supervision and invaluable assistance guided us through every stage of the project's development. His expertise, dedication, and insightful feedback played a pivotal role in refining our research, shaping our methodology, and enhancing our approach. His mentorship not only provided us with invaluable guidance but also instilled in us a deeper understanding of the subject matter. We are profoundly grateful for his unwavering commitment and unwavering support.

Additionally, we sincerely thank our circle of friends and mentors whose wisdom and support enriched our project journey with invaluable advice and encouragement. In conclusion, we express our profound gratitude to everyone who has contributed to our project's success. Your unwavering support, guidance, and encouragement have been invaluable, and we sincerely appreciate your contributions to our achievement.

Table of Contents

Authentication of our work	i
Acknowledgment	ii
Table of Contents	iii
List of Figures	vi
List of Tables	viii
Acronyms	x
Chapter One	1
Introduction	1
1.1 Statement of the Problem	1
1.2 Objectives of the Project	2
1.2.1 General Objective	2
1.2.2 Specific Objectives	2
1.3 Methodology	3
1.3.1 Data Collection	3
1.3.2 System Development Process Model	3
1.3.3 Design Pattern	4
1.3.4 Programming Languages	5
1.4 Tools	5
1.4.1 Hardware Tools	5
1.4.2 Software Tools	5
1.5 Scope and Limitations of the Project	7
1.5.1 Scope	7
1.5.2 Limitations	8
1.6 Significance of Project	8
1.7 Feasibility Study	9
1.7.1 Technical Feasibility	9

1.7.2 Economic Feasibility.....	9
1.7.3 Schedule Feasibility	10
1.8 Risks Assessment	10
1.8.1 Risk.....	10
1.8.2 Assumptions.....	10
1.8.3 Constraint.....	10
1.9 Work Breakdown.....	11
1.9.1 Gantt Chart.....	12
Chapter Two	13
Requirement Analysis and Specification	13
2.1 Introduction	13
2.2 Current System	13
2.2.1 Players of the Existing System.....	15
2.3 Business Rules.....	15
2.4 Proposed System.....	16
2.4.1 Overview.....	16
2.4.2 Functional Requirement	17
2.4.3 Non-Functional Requirements	18
2.4.4 System Model	18
2.4.4.1 Use Case Model.....	18
2.4.4.1.1 Actor Description.....	19
2.4.4.1.2 Use Case Diagram.....	19
2.4.4.1.3 Use Case Description	21
2.4.4.2 Sequence Diagram	34
2.4.4.3 Activity Diagram	38
2.4.4.4 State Chart Diagram.....	43
2.4.4.5 ER Diagram	45

2.4.4.6 Class Diagram.....	47
2.4.4.7 User Interface Prototyping	49
Chapter Three	52
System Design	52
3.1 Introduction	52
3.2 Purpose of the System.....	52
3.3 Design Goals	53
3.4 Current Software Architecture	54
3.5 Proposed Software Architecture.....	54
3.5.1 Subsystem Decomposition.....	56
3.5.2 Component Diagram	58
3.5.3 Deployment Diagram	59
3.5.4 Persistent Data Management.....	60
3.5.5 Detailed Database Design.....	61
3.5.5.1 Relational Table.....	61
3.5.5.2 Database Normalization	63
3.5.5.3 Extended Entity Relation Diagram	66
3.5.5.4 OO-Relational Mapping.....	67
3.5.6 Access Control and Security.....	68
3.5.7 Global Software Control.....	69
3.5.8 Boundary Condition	71
Conclusion.....	73
References	74
Appendix	76
Appendix A: questionnaire for voter.....	76

List of Figures

Figure 1.1: Iterative model	4
Figure 1.2: Workflow Gantt chart.....	12
Figure 2.1: Traditional paper-based voting system.....	14
Figure 2.2: Use case diagram for blockchain-based e-voting system.....	20
Figure 2.3: Sequence diagram for vote casting	34
Figure 2.4: Sequence diagram for vote announcing	35
Figure 2.5: Sequence diagram for generating a report.....	35
Figure 2.6: Sequence diagram for creating a post	36
Figure 2.7: Sequence diagram for viewing voter history	37
Figure 2.8: Sequence diagram for token generation	37
Figure 2.9: Activity diagram for creating an account using a unique token	38
Figure 2.10: Activity diagram for login	39
Figure 2.11: Activity diagram for creating a vote	39
Figure 2.12: Activity diagram for delete vote	40
Figure 2.13: Activity diagram for vote approval	40
Figure 2.14: Activity diagram for the approval of manager account.....	41
Figure 2.15: Activity diagram for cast vote.....	41
Figure 2.16: Activity diagram for announcing voting results	42
Figure 2.17: Activity diagram for viewing voting results.....	42
Figure 2.18: State chart diagram for making payment.....	43
Figure 2.19: State chart diagram for signup and login.....	43
Figure 2.20: State chart diagram for casting vote.....	44
Figure 2.21: State chart diagram for announcing the voting result	44
Figure 2.22: State chart diagram for generating a unique token.....	44
Figure 2.23: ER diagram	46
Figure 2.24: class diagram.....	48
Figure 2.25: Home page	49
Figure 2.26: Pricing page	49
Figure 2.27: Login page	50
Figure 2.28: Vote creating page.....	50
Figure 2.30: Login page layout on a phone.....	51
Figure 2.31: Vote casting on a phone	51

Figure 2.29: Voter home page	51
Figure 3.1: Component diagram	59
Figure 3.2: Deployment diagram	60
Figure 3.3: Normalized ER diagram	66
Figure 3.4: OO-relational mapping.....	67

List of Tables

Table 1.1: Work breakdown	12
Table 2.1: Create an account for the manager use case description	21
Table 2.2: Login use case description	22
Table 2.3: Create an account using a unique token use case description.....	23
Table 2.4: Make payment use case description	23
Table 2.5: Create a voter account use case description.....	24
Table 2.6: Approve manager account use case description.....	24
Table 2.7: Manage their account setting use case description.....	25
Table 2.8: View notification use case description	26
Table 2.9: Generate a token use case description	26
Table 2.10: Generate report use case description	27
Table 2.11: Create vote use case description.....	27
Table 2.12: Edit vote use case description	28
Table 2.13: Approve created, edited, and deleted vote use case description	28
Table 2.14: Delete vote use case description.....	29
Table 2.15: Manage staff and candidate use case description.....	29
Table 2.16: Announce result use case description	30
Table 2.17: Add voter use case description.....	30
Table 2.18: Cast vote use case description.....	31
Table 2.19: Share feedback use case description.....	32
Table 2.20: View result use case description	32
Table 2.21: View candidate post use case description.....	32
Table 2.22: View history use case description	33
Table 2.23: Promote themselves use case description	33
Table 3.1: Account table	61
Table 3.2: Vote table	61
Table 3.3: Company table.....	62
Table 3.4: Candidate table.....	62
Table 3.5: Feedback table.....	62
Table 3.6: Notification table.....	62
Table 3.7: Audit log table.....	62
Table 3.8: Subscription table	62

Table 3.9: Account table normalization	63
Table 3.10: Company table normalization	64
Table 3.11: Vote table normalization.....	64
Table 3.12: Candidate table normalization.....	65
Table 3.13 Subscription table normalization.....	65

Acronyms

API: Application Programming Interface

APP: Application

AWS: Amazon Web Services

CSS: Cascading Style Sheets

DAPP: Decentralized Application

ER: Entity Relationship

HTML: Hypertext Markup Language

HW: Hardware

IDE: Integrated Development Environment

IPFS: Interplanetary File System

JSON: JavaScript Object Notation

MVC: Model-View-Controller

MVT: Model-View-Template

NoSQL: Not Only Structured Query Language

RDBMS: Relational Database Management System

SQL: Structured Query Language

SW: Software

UI: User Interface

UML: Unified Modeling Language

VS Code: Visual Studio Code

Chapter One

Introduction

Voting is a fundamental tool for decision-making across various domains, including democratic governance, entertainment awards, corporate governance, and organizational settings. It allows individuals to express preferences, influence outcomes, and contribute to collective decision-making, fostering inclusivity and transparency.

In democratic governance, citizens vote to elect representatives for public office, ensuring representation and accountability. Similarly, entertainment awards like the Oscars or Grammys rely on voting to determine winners based on artistic merit, impact, and industry recognition. Corporate governance involves shareholder voting on strategic matters like board elections and mergers, shaping the direction and policies of the company. Within organizations, voting decides internal issues such as policies, budgets, and leadership selection, ensuring collective decision-making and member participation.

In the digital age, e-voting has become increasingly important. Blockchain technology plays a crucial role in enhancing its security and integrity. Blockchain is a decentralized and transparent digital ledger that securely records and verifies transactions across multiple computers. It operates on a peer-to-peer network, ensuring immutability, security, and transparency by linking information into an unalterable chain of blocks using cryptographic algorithms [1].

Integrating blockchain into e-voting systems makes each vote cryptographically secure, ensuring integrity and immutability and preventing manipulation or unauthorized alterations. The decentralized and transparent nature of blockchain provides an auditable trail, addressing the challenges of traditional voting methods. This results in a secure, efficient, and inclusive voting system that accurately reflects the collective will of participants while enhancing transparency and accuracy.

1.1 Statement of the Problem

Voting is a fundamental mechanism for decision-making in various contexts. However, despite its importance, several challenges impede the effectiveness and fairness of voting processes outside electoral contexts.

Traditional paper-based voting systems face several difficulties that undermine the effectiveness and integrity of the voting process. One prominent challenge is the potential for discrepancies and conflicts in tabulating results and votes to count. This can arise due to

errors in manually handling and tallying paper ballots, leading to inaccuracies and disputes over the outcome.

Furthermore, the manual nature of paper-based systems introduces inherent vulnerabilities, including the risk of human error and deliberate manipulation. These factors significantly compromise the validity of the voting procedure and erode trust in the integrity of the results. Additionally, reliance on physical infrastructure and transportation for storing and collecting paper ballots poses considerable logistical challenges. Delays and risks associated with these processes can harm the timely announcement of voting outcomes, potentially undermining public confidence in the electoral process.

Previous attempts at electronic voting systems aimed to address some of the limitations of paper-based systems, but they have faced challenges. One significant concern is the security of electronic voting systems, particularly regarding the integrity and confidentiality of the votes. Instances of hacking, manipulation, and unauthorized access have exposed vulnerabilities in these systems, casting doubt on their reliability and raising questions about the accuracy and trustworthiness of the results. The limitations of existing voting systems emphasize their inconvenience and the need for more efficient alternatives.

Our e-voting system is crucial for overcoming the challenges and limitations of traditional paper-based and existing electronic voting systems. Our system provides a comprehensive solution by utilizing the advantages of blockchain-based voting and real-time functionality [2].

1.2 Objectives of the Project

1.2.1 General Objective

The main objective of this project is to develop a secure and transparent blockchain-based voting system that is accessible through user-friendly web applications.

1.2.2 Specific Objectives

The specific objectives of this project are:

- To identify the main requirements for the system.
- To design a scalable, maintainable, and reliable system.
- To develop a user-friendly voting platform with a simple and intuitive interface.
- To integrate blockchain technology to enhance security.
- To increase trust and confidence by utilizing audit mechanisms.
- To test the system to see if it meets all the requirements and is bug-free.

- To enable voters to cast their votes securely and prevent tampering, manipulation, or unauthorized access to voting data.

1.3 Methodology

1.3.1 Data Collection

For this project, we have used three main data collection tools to gain a deep understanding of how voting works and what the requirements are:

Document analysis: Document analysis is pivotal in extracting meaningful insights from various data sources to inform decision-making and guide the design process. Through data analysis, designers can uncover patterns, trends, and correlations within the data that provide valuable information about user behavior, system performance, and other relevant factors. So, we analyzed past research or projects done by other students [3].

Questionnaires: Since a group of people does voting, it is more suitable to collect data from a group of people, which is why we chose questionnaires. We have designed structured questionnaires using the Google Forms platform for distribution and result data analysis. This method allowed us to collect quantitative data on participants' voting perceptions, including their opinions on choices, issues, and the overall voting experience. Also, we have asked individuals who had previously created a voting system.

Observations: We have systematically observed and documented behaviors, events, or phenomena related to voting. This approach provided qualitative data and allowed us to understand better the context being observed. Our team conducts observations, carefully noting the interactions between voters and election officials, the overall dynamics of the voting process, and any noteworthy circumstances or challenges that arise. By directly observing these events, we have captured valuable real-time information, contextual details, and subtle cues contributing to our understanding of participants' voting experiences, perceptions, and behaviors.

1.3.2 System Development Process Model

We will be using the iterative model for the system development process. Iterative System Development breaks projects into small segments called iterations, which undergo planning, designing, coding, testing, and evaluation cycles to achieve gradual improvements.

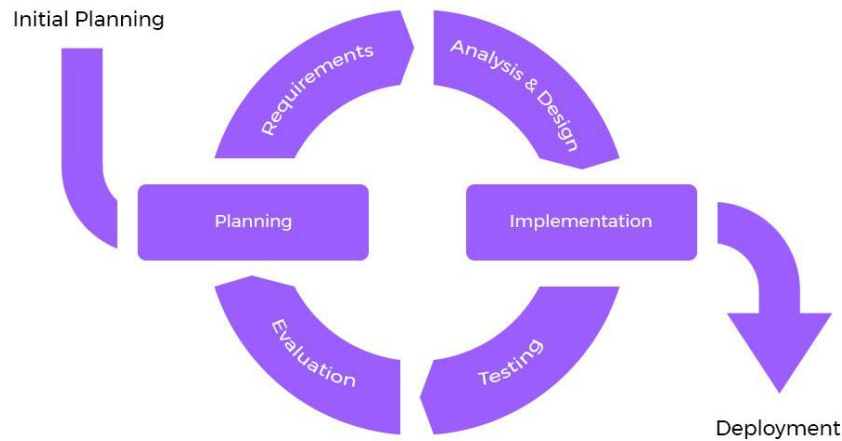


Figure 1.1: Iterative model

There are several reasons why choosing an iterative development model can be beneficial:

- **Flexibility:** Allows for changes and adaptations throughout the development process.
- **Risk Management:** Identifies and addresses potential risks early on.
- **Early Feedback:** Provides opportunities for early and continuous feedback.
- **Improved Quality:** Results in higher quality outcomes through frequent testing and refinement.
- **Stakeholder Engagement:** Keeps stakeholders engaged and informed.
- **Predictable Progress:** Offers precise progress tracking through regular deliverables.

1.3.3 Design Pattern

Design patterns are reusable solutions to common software design problems, serving as adaptable blueprints for addressing specific challenges.

The primary purpose of design patterns is to promote code reusability, maintainability, and scalability while improving the overall quality of software. Here are the design patterns we used:

- **MVT** is a pattern commonly used in web frameworks like Django. In MVT, the Model represents the application's data and business logic. Views represent the presentation layer and handle user interactions. Instead of using Controllers like in MVC, MVT uses Templates. Templates generate the final HTML output by combining data from the Model with presentation markup. Unlike Controllers in MVC, Templates contain minimal logic and are primarily concerned with rendering HTML [4].

- **MVC** - the model represents the application's data and business logic. It encapsulates data manipulation, storage, and business rules. Views represent the presentation layer, display data to users, and handle user interactions. Controllers act as intermediaries between the Model and the View. They handle user input, process requests, update the Model, and update the View with the latest data [5].

We have opted for these two design patterns due to their inherent benefits in scalability and maintainability. This strategic decision ensures our project's agility in adapting to evolving requirements and its potential for growth over time while maintaining stability. Moreover, the robust ecosystem of resources, libraries, and community support surrounding these patterns serves as a valuable aid and guide throughout the development process, significantly contributing to the success of our project. We will implement MVT for Django and MVC for the node.js framework.

1.3.4 Programming Languages

- Python is a high-level programming language that is less verbose and easy to implement.
- HTML - a standard markup language for creating web pages and applications.
- CSS - style sheet language used to describe the presentation and layout of HTML documents.
- JavaScript - a high-level programming language for building dynamic and interactive web applications.
- Solidity - is a programming language for writing smart contracts on the Ethereum blockchain platform.

1.4 Tools

1.4.1 Hardware Tools

- Computer or laptop - for coding the system's function, designing the system, and testing.
- Mobile phone - we will use mobile phones to test the responsive layout of our system.

1.4.2 Software Tools

We have strategically incorporated these software tools into our development process to enhance efficiency and effectiveness. Each tool plays a specific role in our workflow, contributing to the overall success of our software development.

Version Control Systems:

- Git: Git is a distributed version control system that tracks source code changes during software development.
- GitHub: GitHub is a web-based platform and version control system that allows developers to collaborate on projects, host code repositories, and manage software development workflows.

Development Environments and Editors:

- Web Storm: Web Storm is an integrated development environment (IDE) specifically designed for web development.
- VS Code: VS Code is a lightweight yet powerful code editor developed by Microsoft.
- PyCharm: PyCharm is an integrated development environment (IDE) designed explicitly for Python development.

Web Development Tools:

- Web browser: Web browsers are used for testing, debugging, and previewing web applications during development.
- React: React is a JavaScript library for building user interfaces. It allows you to create reusable UI components and build complex UIs using a component-based architecture.
- Node.js: Node.js is a JavaScript runtime environment that allows you to run JavaScript code on the server side. It is chosen for backend development due to its non-blocking I/O and rich ecosystem capability.
- Django: Django is a high-level Python web framework that simplifies web development by providing built-in features and conventions. It is chosen for its focus on rapid application development and scalability.
- Tailwind - is a utility-first CSS framework that provides predefined utility classes for styling HTML elements.

Database Management Systems:

- PostgreSQL: PostgreSQL is an open-source relational database management system (RDBMS) known for its reliability, robustness, and extensibility.
- MongoDB: MongoDB is a popular NoSQL database that stores data in flexible, JSON-like documents.

Diagramming and Design Tools:

- Lucid Chart: Lucid Chart is a designing software with many built-in templates for UML diagrams and other diagrams.

- Miro.com: Miro.com is a website that can design UML diagrams.
- Figma: Figma is a web-based designing app that can be used to build prototypes and UI interfaces.
- SmartDraw: SmartDraw is a diagramming software application that provides tools for creating diagrams, including flowcharts, mind maps, and other visual representations.
- draw.io: Draw.io is versatile online diagram software for creating various diagrams, including use case and activity diagrams.

API Development and Testing Tools:

- Postman: Postman allows us to design, test, and debug APIs quickly and efficiently, with features like automated testing, mock servers, and API monitoring.
- Ethereum Blockchain: Ethereum Blockchain is a decentralized platform that enables the development of smart contracts and decentralized applications (dApps).

Deployment and Containerization Tools:

- AWS: AWS is used to deploy our web applications databases and test servers.
- Docker: Docker is a containerization platform that allows you to package, distribute, and run applications in lightweight, isolated containers.

Project Management Tools:

- Linear: Linear is a popular project management tool for small team projects.
- Eraser.io: Eraser.io is a website that helps organize software projects.

1.5 Scope and Limitations of the Project

1.5.1 Scope

- **Geographical Limitation:** The system will not have any geographical restrictions if internet access exists.
- **UI Languages:** The system supports two UI languages - Amharic and English.
- **Secure Voting:** The system uses blockchain technology to ensure the security and integrity of votes. Votes are encrypted to prevent tampering and unauthorized access.
- **Real-time Results:** The system provides immediate access to voting outcomes, enabling voters and relevant parties to view the results in real time.
- **User-friendly and Easy UI Interface:** The system features a user-friendly and intuitive interface, ensuring voters can easily navigate and interact with website interfaces.
- **Device Compatibility:** The system is accessible through a mobile and a website but is not compatible with offline devices.

- **Data Privacy:** Robust data protection measures will be implemented to ensure the confidentiality of voter information.
- **Authentication and Access Control:** The system incorporates a robust authentication and access control module. It verifies user credentials, implements secure login procedures, and enforces access rights based on user roles and permissions.
- **Ballot Management:** The system creates, configures, and distributes electronic ballots to eligible voters. It ensures the accuracy and integrity of the ballot content, manages different types of voting options, and handles any necessary validation or verification processes associated with the ballots.
- **Audit and Logging:** It maintains a detailed log of user interactions, system operations, and critical events for auditing, monitoring, and troubleshooting purposes. Also, it enables administrators to track system activities, detect potential security breaches, and investigate any irregularities or discrepancies that may occur during the voting process.

1.5.2 Limitations

1. **Limited Accessibility:** The voting system's online nature may restrict access for individuals in rural areas with inadequate internet infrastructure or connectivity, impacting their ability to participate in the voting process.
2. **Language Exclusivity:** This system supports only two languages, English and Amharic, which may limit accessibility for voters who do not speak either language.

1.6 Significance of Project

This project is essential as it aims to transform voting processes and address significant challenges. Here are the primary impacts:

- **Improving Accessibility:** The project ensures that voting becomes effortless and accessible by creating a user-friendly electronic voting system. It breaks down barriers imposed by geographical limitations and administrative complexities, enabling broader participation and promoting inclusivity.
- **Streamlining Voting Processes:** This project facilitates and simplifies voting by integrating blockchain technology and real-time e-voting. It eliminates the inefficiencies of traditional paper-based methods, reducing administrative burdens and saving valuable time.

- **Ensuring Security and Integrity:** Incorporating blockchain technology provides a decentralized and transparent ledger, safeguarding the security and integrity of voting data. This ensures that votes cannot be tampered with, manipulated, or compromised, instilling trust and confidence among participants.
- **Promoting Transparency and Trust:** The project prioritizes transparency by implementing mechanisms that provide visibility of the voting process. Through blockchain and real-time e-voting, this transparency creates an auditable trail, building trust among voters and strengthening the reliability of the outcomes.
- **Facilitating Informed Decision-Making:** This project promotes informed decision-making by overcoming limited communication and restricted access. It provides a user-friendly platform with clear instructions, making it easier for voters to participate and contribute to the democratic process.
- **Versatility for Diverse Voting Scenarios:** This comprehensive electronic voting system caters to various voting scenarios, including national elections, entertainment awards, corporate governance, and organizational decisions. Its adaptability and flexibility make it a valuable tool across multiple domains.

1.7 Feasibility Study

A feasibility study is an assessment conducted to determine the practicality of a project, while our feasibility studies evaluate the project's practicality, timeliness, and financial considerations.

1.7.1 Technical Feasibility

The project is feasible based on the availability of the necessary technical resources, including knowledge of blockchain technology, hardware, and software. These resources, combined with the abilities of our skilled team, provide a solid foundation for successful implementation. The team's knowledge of developing user-friendly interfaces and implementing robust security protocols further improves the project's feasibility. By highlighting our team's technical skills and alignment with the chosen technologies, it becomes evident that the project is technically feasible and well-positioned for successful execution.

1.7.2 Economic Feasibility

We have achieved economic feasibility in our voting system project by implementing effective cost-saving measures. Using free, open-source software tools eliminates the need for software expenses, while our reliance on online data gathering minimizes paper usage and

related costs. We can optimize resource allocation and ensure the project's financial viability by adopting cost-effective software and utilizing our existing hardware resources.

1.7.3 Schedule Feasibility

With the strategic implementation of project management techniques and the availability of necessary resources, we can complete the project within the specified timeframe. Through thorough planning and precise resource allocation, we have ensured that the project is considered time-feasible and achievable within the given deadline.

1.8 Risks Assessment

1.8.1 Risk

The potential risks we may encounter include:

- Their changes can affect our project as we develop our system based on the Ethereum blockchain.
- Some of our team members may be unable to participate as needed due to personal issues.
- Finishing the project on time might be challenging because the project's scope is immense.
- Integrating the project with Amazon Web Services (AWS) may introduce additional complexity and potential risk.

1.8.2 Assumptions

The assumptions we made for this project are:

- We assume the Ethereum blockchain will only change significantly once we finish the project.
- We are assuming that AWS services will continue to be reliable.
- We also assume that our team members will participate equally and without personal problems.
- Despite the project's enormous scope, we assume it can be completed within the designated timeframe with sufficient resources and effective management.

1.8.3 Constraint

Some of the constraints of this project include:

- Given that we will be concurrently undertaking the project alongside learning activities, our time for project completion may be limited.
- Incorporating blockchain technology introduces budget constraints, which can limit resources due to the costs associated with executing transactions on the blockchain.

1.9 Work Breakdown

This breakdown outlines the significant tasks in developing and deploying a blockchain voting system, from initial planning and design to ongoing implementation. Each task may involve multiple sub-tasks and iterations to ensure the system's successful implementation.

#	Activity	Duration (in Days)	Deliverable
1	Requirement	25	• Scope • Requirement specification • Feasibility study
	Project analysis and identification	5	
	Feasibility study	20	
2	Planning	25	• Resource allocation • Project schedule • Risk assessment document
	Estimations	10	
	Scheduling	5	
	Risk analysis	10	
3	Analysis And Design	45	• System Architecture Diagrams • Data model diagrams • Use case diagrams • User interface wireframes
	Use case modeling	10	
	UI prototype	15	
	Sequence diagram	7	
	Class diagram	3	
	ER diagram	4	
	Activity diagram	6	
4	Implementation	70	• Database and Database queries • Smart contracts • Backend APIs • Frontend code
	Smart contract	20	
	Database	10	

	API	20	
5	Testing	20	• Test plan document • Test cases • Test environment setup • Defects and issues found
	Unit testing	8	
	Integration testing	4	
	Functional testing	2	
	Security testing	6	
5	Evaluation	5	
6	Deployment	5	• A fully functional system deployed to AWS

Table 1.1: Work breakdown

1.9.1 Gantt Chart

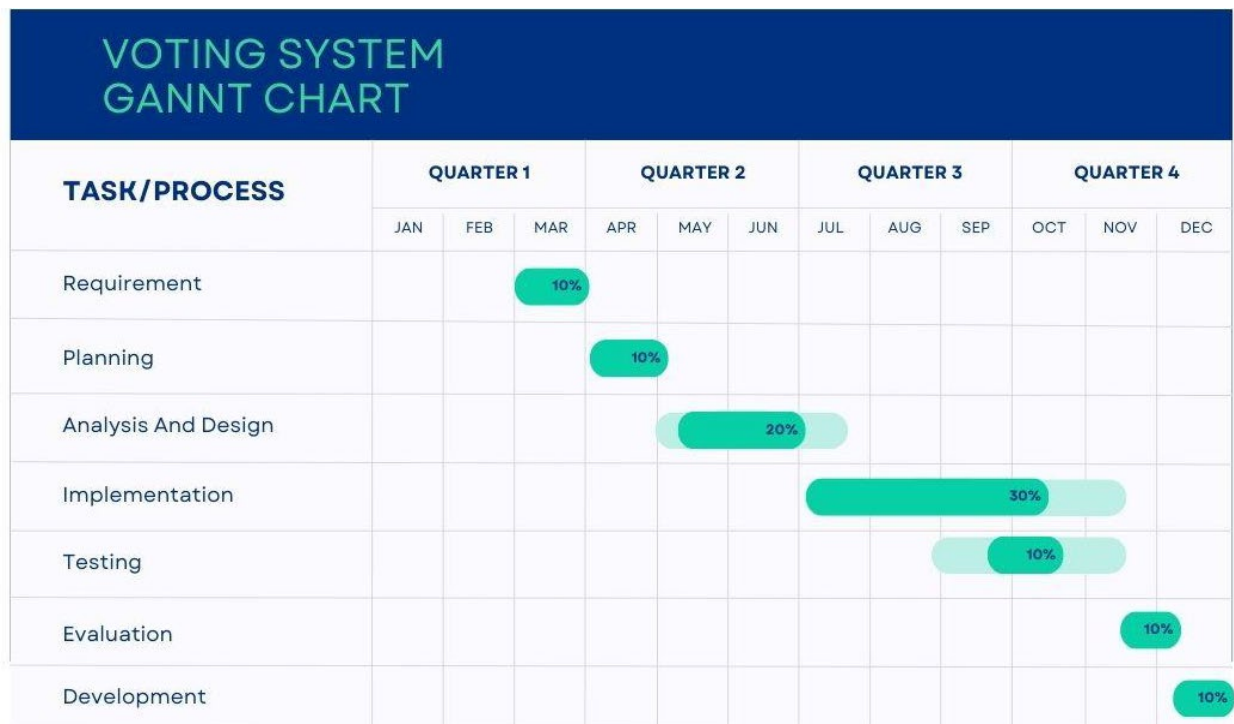


Figure 1.2: Workflow Gantt chart

Chapter Two

Requirement Analysis and Specification

2.1 Introduction

This chapter analyzes and defines the proposed system's business requirements. We aim to thoroughly understand the system's necessary functionalities and characteristics and ensure it meets the desired objectives.

Next, we define the proposed system's functional and non-functional requirements. Functional requirements outline the specific actions, behaviors, and functionalities the system must possess to fulfill its intended purpose. Non-functional requirements encompass performance, security, usability, and scalability, contributing to the system's overall quality and effectiveness.

Additionally, we present system models that provide visual representations of the proposed system's structure, interactions, and components. These models aid in understanding the system's architecture and facilitate effective communication among stakeholders.

2.2 Current System

The current voting system includes various methods and technologies used in democratic processes, entertainment contexts, and organizational settings. It is important to note that there currently needs to be an integrated system for all types of voting, whether traditional paper-based or electronic. Different areas and organizations may adopt different voting methods and technologies based on their needs and resources.

Traditionally, paper-based voting has been the primary method used in governmental elections and organizational decision-making processes. In this system, eligible voters, typically citizens above a certain age and meeting residency requirements, have the right to participate in the voting process. The criteria for voting and participation are usually established by laws and regulations specific to each jurisdiction or organization.

When a voting event occurs, eligible voters are provided with physical ballots on which they mark their preferences by indicating their choices with a pen. These ballots are collected by election officials stationed at designated polling stations or voting centers. The process may involve verifying the eligibility of voters through identification documents or voter registration lists.

Once the voting period concludes, election officials manually count the collected ballots. These officials are typically trained personnel responsible for ensuring the accuracy and integrity of the vote count. Any issues or disputes that arise during the voting process, such as

contested ballots or allegations of irregularities, are addressed according to established procedures. Election officials and designated legal authorities handle these matters based on laws and regulations that govern the electoral process. Standard practices include conducting investigations, adjudicating disputes, and potentially recounting votes if necessary.

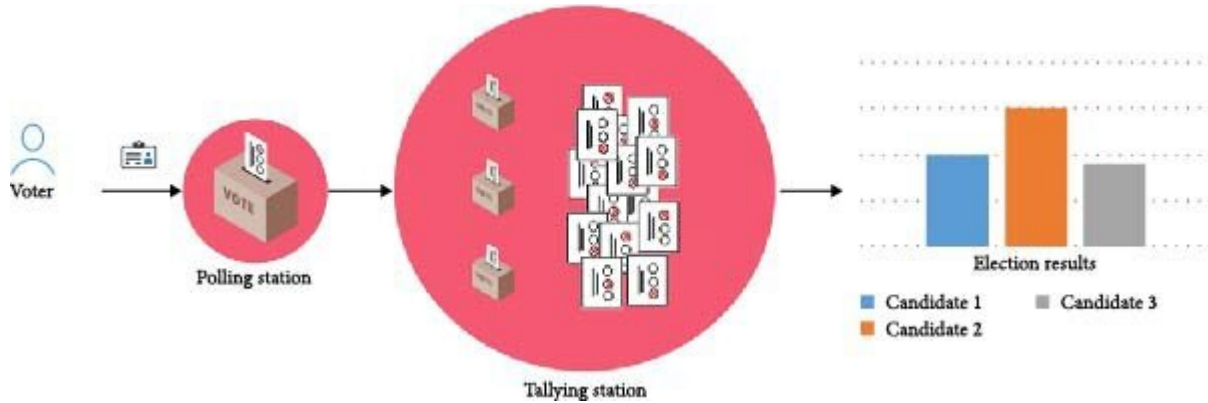


Figure 2.1: Traditional paper-based voting system

In addition to the traditional paper-based voting system, electronic voting (e-voting) systems have gained popularity, particularly in entertainment settings. These systems use digital technologies like the web or mobile applications, allowing viewers or participants to vote electronically. The criteria for voting and participation in entertainment-based e-voting systems typically vary depending on the show or competition.

Viewers who meet specific eligibility requirements, such as age restrictions or residency criteria, can participate. The exact criteria for voting and participation are usually outlined and communicated by the organizers or broadcasters of the respective entertainment program. The counting of votes in electronic voting systems is typically automated. The ballots are transmitted and stored in a central database. The current entertainment voting system uses algorithms and software to tabulate the results accurately and efficiently. The process is often overseen by a combination of technical experts, election officials, and independent auditing firms to ensure the integrity and accuracy of the tabulation process [6].

Organizational voting also plays a distinct role within the current voting landscape. Unlike traditional and entertainment voting systems, organizational voting uses various methods, such as in-person voting during meetings, mail-in ballots, or online voting platforms. The choice of voting method depends on factors such as the organization's size, geographical dispersion of participants, and desired levels of security and transparency.

2.2.1 Players of the Existing System

The key participants who play distinct roles contributing to the overall functionality and efficiency of the system are:

- **Voters** actively participate by casting votes and influencing the election's outcome or decision. Eligible voters vary based on the specific scenario, such as citizens in national polls, employees in organizational voting, or eligible participants in entertainment voting.
- **Candidates** are another crucial group in the voting system. They seek to be elected or chosen for a specific position or office. They actively engage with the voters, campaigning and presenting their platforms or ideas to gain support and secure their votes.
- **Party heads** are influential leaders who shape party platforms, manage their candidates, and represent political parties in national elections.
- **Election officials:** These officials organize and conduct the voting process. They carefully follow voting procedures, verify voters' eligibility, manage polling stations or voting centers, and accurately count the votes.
- **Legal authorities**, such as election commissions, enforce the legality and compliance of the voting process. They establish the necessary rules and regulations, handle any legal disputes or challenges that may arise, and ensure that the voting process aligns with the law.
- **Independent observers:** These individuals closely observe the voting process, attentively looking for any irregularities or violations that may undermine the election's fairness.
- **Organizers or broadcasters** of TV shows, competitions, or other entertainment events have a role to play. They provide the platform and mechanisms for viewers or participants to vote. These organizers or broadcasters facilitate the voting process, enabling the determination of outcomes in entertainment-based contexts.

2.3 Business Rules

These business rules are directives that govern and guide the operations, definitions, and constraints within a voting environment. They define or constrain some aspects of the voting, ensuring consistency and compliance with the company's objectives and policies.

- **Rule 1:** Each eligible voter can cast only one vote in the election.

- **Rule 2:** Only eligible voters who meet the specified criteria (e.g., age) can participate in the voting process.
- **Rule 3:** Only citizens of a particular country can vote in elections.
- **Rule 4:** To participate in decision-making or other kinds of voting in a company, the individual should be a staff member. Different criteria may apply based on the type of vote (e.g., board vote, stakeholder vote).
- **Rule 5:** For closed votes, the result will be tallied after the voting duration has ended and announced by the administrator or the user who created the voting.
- **Rule 6:** Voter anonymity and privacy must be maintained throughout voting.
- **Rule 7:** The voting system must provide transparency, allowing stakeholders to verify the integrity and fairness of the election process.
- **Rule 8:** For verification-enabled voting, users must upload a valid identification document, such as a driver's license or ID card, and a photo of themselves to verify their identity.
- **Rule 9:** If a user encounters any issues while voting, our company staff will be on hand to resolve the problem.
- **Rule 10:** Eligible participants are sourced from the organization for internal voting.
- **Rule 11:** The voting system must comply with relevant legal and regulatory requirements, including data protection laws and election regulations.

2.4 Proposed System

2.4.1 Overview

The proposed Secure Voting System modernizes voting by prioritizing efficiency, security, and transparency. It simplifies voter registration with an intuitive interface, emphasizes data security through validation checks, and provides real-time feedback on registration status. The user-friendly design accommodates voters of all technical backgrounds, ensuring inclusivity and accessibility for all participants.

Utilizing blockchain technology, the system encrypts and records each vote on a decentralized ledger, ensuring tamper-proof and secure voting records. This transparency allows stakeholders to verify the voting process, fostering trust and eliminating the risk of manipulation and fraud. Real-time voting capabilities offer up-to-date information and results, reducing the time required for result announcements and increasing voter engagement and satisfaction.

Scalability and reliability are critical considerations in the system's design. It handles high volumes of users with robust redundancy and failover mechanisms. Automated announcements and customizable notifications ensure timely updates on crucial election information accessible through a centralized platform. Overall, the Secure Voting System enhances the voting experience for various elections by addressing traditional challenges with advanced technology, promoting trust, transparency, and efficiency in the democratic process.

2.4.2 Functional Requirement

- It allows users to create an account by providing necessary identification information.
- The system allows all users to log in to their respective accounts using their credentials, ensuring controlled access and authentication.
- The system will allow all users to update their personal information, change passwords, and perform other account-related management tasks.
- The system will enable managers to initiate the account creation process and become authorized users by providing the necessary information.
- It allows managers to make payments to cover system-related expenses.
- The system admin reviews and approves or rejects requests for manager account creation.
- It allows managers to generate unique tokens provided to staff members, parties, and candidates to facilitate their account creation process.
- It allows parties, candidates, and authorized staff members to create accounts using a unique token supplied to them.
- The system enables party heads to manage and oversee their candidates efficiently.
- Parties and Candidates can communicate their campaign messages, proposals, or qualifications to the voters.
- It grants staff members the authority to create new voting events and edit or delete existing votes, allowing them to make necessary modifications and remove outdated or irrelevant voting events when required.
- The system allows managers to review and approve or reject votes created, edited, or deleted by staff members, ensuring the accuracy and validity of voting events.
- The system enables voters to review the available options or candidates for each voting event, allowing them to make informed decisions.
- Voters can securely cast their votes for different voting events or candidates.

- If the voting is private, the system allows staff members to add eligible voters to specific voting events, ensuring that only authorized individuals can participate.
- The system allows voters to access and view their voting history. It enables them to provide feedback on the voting system, process, or related aspects, facilitating continuous improvement and addressing concerns.
- If the result of a voting event is ready, staff members are responsible for announcing the results.
- The manager can generate reports based on voting data and system activities.
- Users with different roles can receive notifications regarding important updates, events, or actions related to their voting processes.
- The system allows voters and candidates to access and view the results of completed voting events or elections.

2.4.3 Non-Functional Requirements

- The system should handle a large volume of concurrent users and transactions during peak voting periods while ensuring minimal response times for user interactions such as casting votes and viewing election results.
- It should scale horizontally and vertically to accommodate growth in users, elections, and voting events, maintaining high availability and resilience to failures through redundancy and failover mechanisms.
- Adherence to industry best practices and compliance standards for data security, privacy, and confidentiality is essential, including robust security measures like multi-factor authentication, encryption, and access controls to protect sensitive information.
- Additionally, the system should maintain comprehensive audit logs and transaction records for forensic analysis and regulatory compliance.
- The user interface should follow usability principles and accessibility guidelines for inclusivity, supported by user documentation and training materials for effective navigation.

2.4.4 System Model

2.4.4.1 Use Case Model

A use case model is a comprehensive system representation that includes a graphical depiction of user interactions and use case actor descriptions and use case description tables. It provides a complete understanding of the system's functionality, roles, and relationships.

2.4.4.1.1 Actor Description

- **System Admin:** A person responsible for the overall management and maintenance of the voting system. They can approve manager accounts and monitor the system's performance to ensure smooth operation.
- **Voter:** An individual who has the right to participate in the voting process. Voters can create an account, cast their votes, view their voting history and results, and provide feedback within the voting system. Voters' eligibility varies depending on the scenario, such as citizens in national elections, employees in organizational voting, or eligible participants in entertainment voting.
- **Authorized company staff:** A person with specific privileges the organization grants to manage the voting system. They can create, edit, delete, and announce votes. Additionally, they can add voters for private voting within the system.
- **Manager:** The main administrator of the company's voting system. Managers oversee the voting process, including managing staff, parties, and candidates. They approve created, edited, and deleted votes within the system. Additionally, they generate reports.
- **Candidate:** A person who participates in voting as a potential recipient of votes. Candidates promote themselves and seek votes within the voting system, aiming to secure support from voters.
- **Party head:** A person representing a political party within the voting system, specifically during national elections. They are responsible for managing candidates and promoting their party.

2.4.4.1.2 Use Case Diagram

A use case diagram is a visual tool that shows how users interact with a system to accomplish tasks or goals. It uses stick figures to represent users and ovals to represent tasks. It helps define system requirements and communicate its behavior to stakeholders [7].

Blockchain-Based E-Voting System

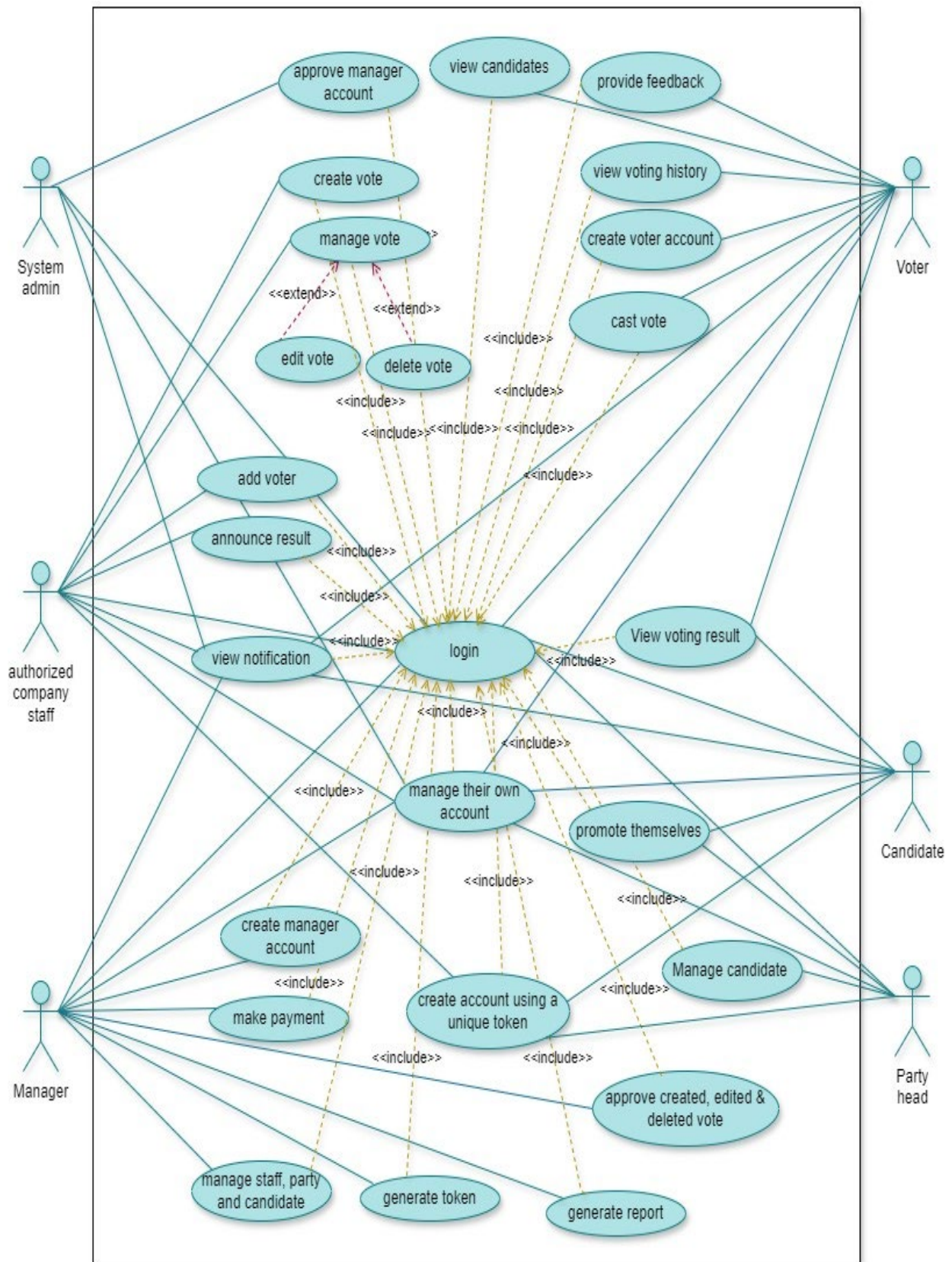


Figure 2.2: Use case diagram for blockchain-based e-voting system

2.4.4.1.3 Use Case Description

Use Case Number	UC01
Use Case Name	Create an account for the manager.
Participating Actors	• Manager, System admin
Preconditions	• Letter of advice from a company
Main Flow	1. Click Create account. 2. The system displays the Create Account form. 3. The Manager fills out the form and attaches a letter of advice. 4. Click Submit. 5. Verify competence of data and save. 6. Display a message that we will respond shortly. 7. Use case ENDS.
Alternative Flow	In step 5, if the data is incomplete or incorrect, the system will show an error message and prompt the user to rewrite it.
Post Condition	The system will display the home page, and the system administrator will be notified.
Extend	None
Include	UC07

Table 2.1: Create an account for the manager use case description

Use Case Number	UC02
Use Case Name	Login
Participating Actors	• System admin, Manager, Company staff, Candidate, Voter
Preconditions	The user should be registered.

Main Flow	1. The user clicks the login button. 2. The system displays the login form. 3. The user enters his Email and password. 4. Click submit. 5. The system verifies the correctness and completeness of the entered data. 6. The system checks if a user exists with that credential. 7. Displays home page. 8. Use case ENDS.
Alternative Flow	In step 6, if no user exists with that credential, the system lets the user try again. In step 6, if the user has enabled two-factor authentication, the system will prompt the user to enter the two-factor authentication.
Post Condition	The system will display the home page.
Extend	None
Include	None

Table 2.2: Login use case description

Use Case Number	UC03
Use Case Name	Create an account using a unique token
Participating Actors	• Company staff, Candidate, Election commission
Preconditions	• A token must be generated
Main Flow	1) After receiving the link, the user will be redirected to the account creation page with a form. 2) After the new user fills out the form, click submit. 3) Verify completeness of data and save. 4) Display the “Account Successfully Created” Message. 5) Use case ENDS.
Alternative Flow	In step 3, if the user does not complete all the necessary information, an error message will appear, and the user will be prompted to enter again.
Post Condition	The system will display the home page.
Extend	None

Include	UC09
---------	------

Table 2.3: Create an account using a unique token use case description

Use Case Number	UC04
Use Case Name	Make a Payment
Participating Actors	● Manager, System admin
Preconditions	● A manager with an account
Main Flow	1. The manager will navigate to the purchases tab. 2. The system will display the types of plans available. 3. The manager will choose his desired plan. 4. The system will navigate to the payment tab and display payment options. 5. The manager will choose a payment option. 6. The manager will make the payment. 7. The system admin confirms that the payment has been made. 8. Use case ENDS.
Alternative Flow	In step 7, the system admin will contact the manager if the payment has yet to be made.
Post Condition	The system will display the home page.
Extend	None
Include	UC02

Table 2.4: Make payment use case description

Use Case Number	UC05
Use Case Name	Create a voter account.
Participating Actors	● Voter
Preconditions	● None
Main Flow	1. The user will click Sign up. 2. The system will display a create account form. 3. The user appropriately fills out the form. 4. Click submit. 5. The system checks for incorrectness and incompetence. 6. Send an email or text message to verify the user.

	7. The user enters the sent code. 8. Display the “Account Successfully Created” Message. 9. Use case ENDS.
Alternative Flow	In step 5, if the user does not complete all the necessary information, it shows an error message and prompts the user to enter again. In step 7, if the user enters an incorrect code, the system will prompt them to enter it again.
Post Condition	The system will display the home page.
Extend	None
Include	UC02

Table 2.5: Create a voter account use case *description*

Use Case Number	UC06
Use Case Name	Approve manager account.
Participating Actors	System admin
Preconditions	Account creation initiated
Main Flow	1. After the manager creates an account, the system will notify the system admin. 2. The system admin checks that the information entered by the company is correct and that the document is legitimate. 3. Navigate to the user tab. 4. The system will display a list of users. 5. The system admin selects the user and approves his account
Alternative Flow	In step 2, the system admin would request a resubmission if the information entered is incorrect.
Post Condition	An account will be created for the manager.
Extend	None
Include	UC02

Table 2.6: Approve manager account use case *description*

Use Case Number	UC07
-----------------	------

Use Case Name	Manage their account setting.
Participating Actors	System admin, Manager, Company staff, Candidate, Voter, Election commission, Party head
Preconditions	A user with an account
Main Flow	- The user will navigate to the account tab. - Then click Edit account. - The system will display a form. - The user changes the desired information. - Click Update. - The system checks the entered information. - Send a confirmation email. - The system updates the user information. - Use case ENDS.
Alternative Flow	In step 7, if the user doesn't confirm the change by clicking the email, the system will not change the user's data.
Post Condition	The system will display the home page.
Extend	None
Include	UC02

Table 2.7: Manage their account setting use case description

Use Case Number	UC08
Use Case Name	View notification
Participating Actors	System admin, Manager, Company staff, Candidate, Voter, Election commission, Party head
Preconditions	A user with an account
Main Flow	- After the system or another user has changed, the system will email a specified user. - The specified user will receive the notification. - When the user clicks the notification. - The system displays the notification. - Use case ENDS.
Alternative Flow	None
Post Condition	The system will display the notification.

Extend	None
Include	UC02

Table 2.8: View notification use case description

Use Case Number	UC09
Use Case Name	Generate a token
Participating Actors	• Manager
Preconditions	• A manager with an account
Main Flow	1. The manager will navigate to the user's tab. 2. Then click Add a User. 3. Then, enter the new user's phone number or email. 4. Choose their role in the company. 5. Click Generate. 6. The manager selects a media to share the unique token. 7. The system will generate a link with a unique token and share it with the user through the selected media. 8. Use case ENDS.
Alternative Flow	None
Post Condition	The system will display the home page.
Extend	None
Include	UC02

Table 2.9: Generate a token use case description

Use Case Number	UC10
Use Case Name	Generate report
Participating Actors	• Manager
Preconditions	• A manager with an account
Main Flow	1. Report Generation. 2. The system retrieves the required data. 3. Select the desired report type. 4. Processes the data and generates the report. 5. Use case ENDS.

Alternative Flow	In step 2, if the system cannot retrieve the required data, it displays an error message indicating the unsuccessful retrieval.
Post Condition	The system notifies the voters and candidates about the availability of the report.
Extend	None
Include	UC02

Table 2.10: Generate report use case description

Use Case Number	UC11
Use Case Name	Create vote
Participating Actors	• Company staff
Preconditions	• A company with a subscription
Main Flow	1. The company staff navigates to the vote page. 2. Clicks create votes. 3. The system displays a form for vote creation. 4. The company staff fills out the form. 5. Clicks submit. 6. Use case ENDS.
Alternative Flow	In step 5, the entry will be discarded if the user doesn't click submit.
Post Condition	A published vote
Extend	None
Include	UC02

Table 2.11: Create vote use case description

Use Case Number	UC12
Use Case Name	Edit vote
Participating Actors	• company staff
Preconditions	• A published vote or a draft vote

Main Flow	1. The company staff selects the vote to be edited. 2. The system will display a form. 3. The company staff edits the form. 4. The company staff clicks update. 5. Use case ENDS.
Alternative Flow	In step 5, the entry will be discarded if the user doesn't click submit.
Post Condition	A modified vote
Extend	None
Include	UC02

Table 2.12: Edit vote use case description

Use Case Number	UC13
Use Case Name	Approve created, edited, and deleted vote
Participating Actors	• Manager
Preconditions	• A published vote or a draft vote
Main Flow	1. The manager will receive a notification to verify the vote. 2. The manager authorizes the modification. 3. The modification will be published.
Alternative Flow	In step 2, if the manager didn't authorize the modification, the vote will be saved as a draft without modification.
Post Condition	A Modified vote
Extend	None
Include	UC02

Table 2.13: Approve created, edited, and deleted vote use case description

Use Case Number	UC14
Use Case Name	Delete vote
Participating Actors	• company staff
Preconditions	• A published vote or a draft vote

Main Flow	1. The company staff selects the vote to be deleted. 2. The system will display a confirmation message. 3. The company staff confirms. 4. Use case ENDS.
Alternative Flow	In step 3, the entry will be discarded if the user didn't click submit.
Post Condition	A deleted vote.
Extend	None
Include	UC02

Table 2.14: Delete vote use case description

Use Case Number	UC15
Use Case Name	Manage staff and candidate
Participating Actors	• Manager
Preconditions	• A manager account
Main Flow	1. Navigate to the staff and candidates tab. 2. The system displays a list of staff and candidates. 3. The manager selects a staff or candidate. 4. The system displays the selected user information. 5. The manager makes the desired changes. 6. Click save. 7. Use case ENDS.
Alternative Flow	None
Post Condition	A modified staff or candidate account.
Extend	None
Include	UC02

Table 2.15: Manage staff and candidate use case description

Use Case Number	UC16
Use Case Name	Announce result
Participating Actors	• company staff
Preconditions	• A published vote

Main Flow	1. The company staff selects the vote. 2. The system will display a description. 3. The company clicks the announce button. 4. The system will announce the results to the candidates and voters. 5. Use case ENDS.
Alternative Flow	None
Post Condition	The vote will be announced.
Extend	None
Include	UC02

Table 2.16: Announce result use case description

Use Case Number	UC17
Use Case Name	Add voters
Participating Actors	• company staff
Preconditions	• A published vote or a draft vote
Main Flow	1. The company staff selects to add voters. 2. The system will display the ability to add or upload from a file individually. 3. The company staff chooses. 4. The system will display a form corresponding to the company staff's previous selection. 5. The company staff fills out the form and clicks add. 6. The system notifies the voters. 7. Use case ENDS.
Alternative Flow	In step 6, if the system fails to notify the voters, it will inform the company staff.
Post Condition	A published vote.
Extend	None
Include	UC02

Table 2.17: Add voter use case description

Use Case Number	UC18
-----------------	------

Use Case Name	Cast vote
Participating Actors	• Voter
Preconditions	• A public vote or a vote with authorization to vote
Main Flow	1. The voter chooses a vote. 2. The system displays the vote with options. 3. The voter chooses from the options. 4. The system displays a confirmation message. 5. The voter agrees. 6. The system will save the voter option. 7. Use case ENDS.
Alternative Flow	In step 5, the system will display the vote options again if the voter disagrees.
Post Condition	A voter will cast an option.
Extend	None
Include	UC02

Table 2.18: Cast vote use case description

Use Case Number	UC19
Use Case Name	Share feedback
Participating Actors	• Voter
Preconditions	• A public vote or a vote with authorization to vote
Main Flow	1. The voter chooses a vote that they already voted for. 2. The system displays the vote. 3. The voter clicks the share feedback button. 4. The system displays a form for feedback. 5. The voter enters their feedback and clicks submit. 6. The system will save the voter feedback. 7. Use case ENDS.
Alternative Flow	None
Post Condition	Shared feedback.
Extend	None
Include	UC02

Table 2.19: Share feedback use case description

Use Case Number	UC20
Use Case Name	View result
Participating Actors	• Voter
Preconditions	• A public vote or a vote with authorization to vote
Main Flow	1. The voter chooses a vote that they already voted for. 2. The system displays the result if it is already announced. 3. Use case ENDS.
Alternative Flow	In step 2, the system displays a message if the vote result isn't announced.
Post Condition	View result
Extend	None
Include	UC02

Table 2.20: View result use case description

Use Case Number	UC21
Use Case Name	View Candidate post
Participating Actors	• Voter
Preconditions	• A candidate post
Main Flow	1. Select a vote. 2. The system displays a vote description. 3. The voter selects candidates. 4. The system displays a list of candidates. 5. The voter chooses a candidate. 6. The system displays the candidate's post. 7. Use case ENDS.
Alternative Flow	In step 6, the system will display a message if the candidate has not posted anything.
Post Condition	View candidate post.
Extend	None
Include	UC02

Table 2.21: View candidate post use case description

Use Case Number	UC22
Use Case Name	View History
Participating Actors	• Voter
Preconditions	• At least one vote
Main Flow	1. The voter navigates to the library tab. 2. The system displays the voter past vote list. 3. The voter selects a vote. 4. The system displays a description of the vote. 5. Use case ENDS.
Alternative Flow	None
Post Condition	View History
Extend	None
Include	UC02

Table 2.22: View history use case description

Use Case Number	UC23
Use Case Name	Promote themselves
Participating Actors	• Candidates, Party head
Preconditions	• Account
Main Flow	1. The candidate navigates to the account page. 2. The system displays the account page. 3. The candidate clicks Create post. 4. The system displays a form. 5. The candidate fills out the form and clicks the post. 6. Use case ENDS.
Alternative Flow	None
Post Condition	Create Post
Extend	None
Include	UC02

Table 2.23: Promote themselves use case description

2.4.4.2 Sequence Diagram

A Sequence Diagram is a type of interaction diagram that shows how objects interact in a particular sequence of time. It details the flow of messages between different objects and actors to accomplish a specific task or part of the functionality within a system [8]. We have used sequence diagrams to analyze the system.

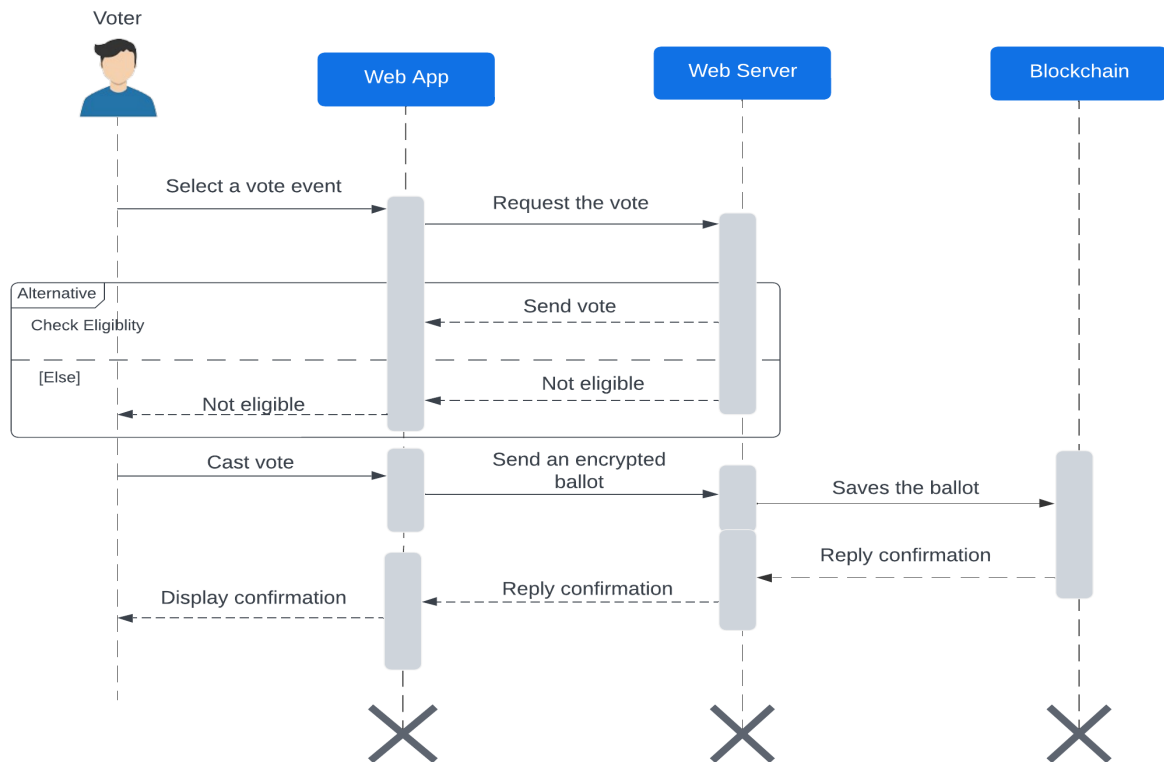


Figure 2.3: Sequence diagram for vote casting

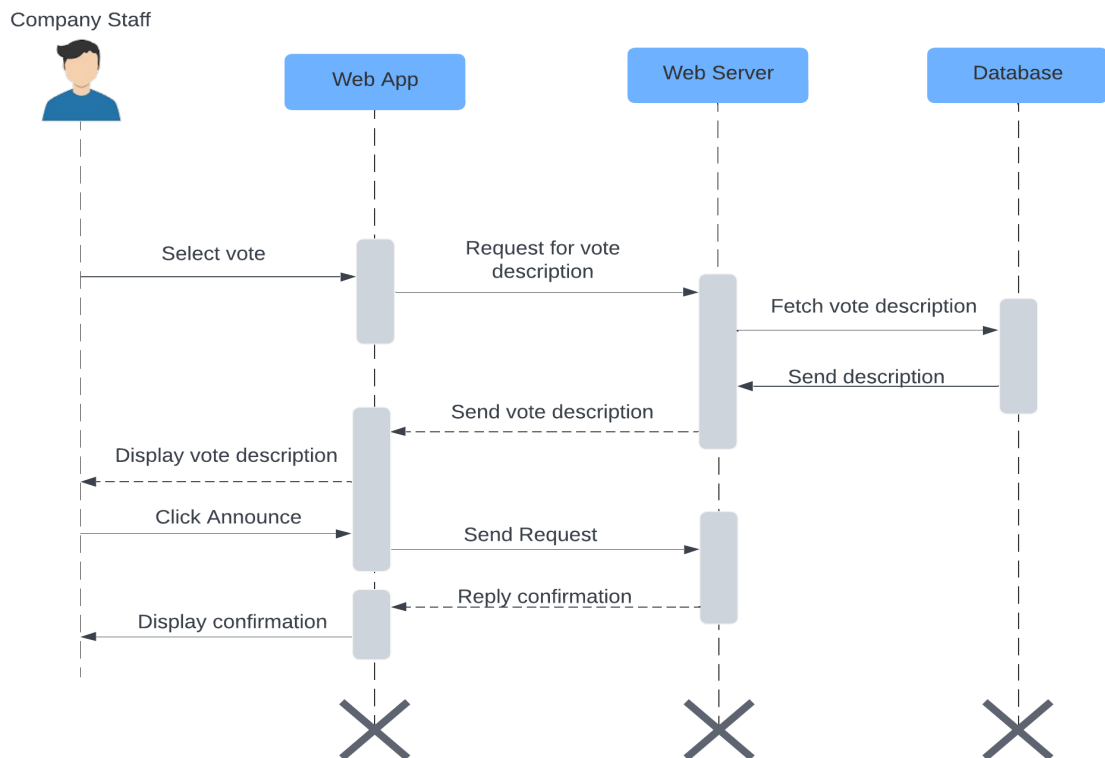


Figure 2.4: Sequence diagram for vote announcing

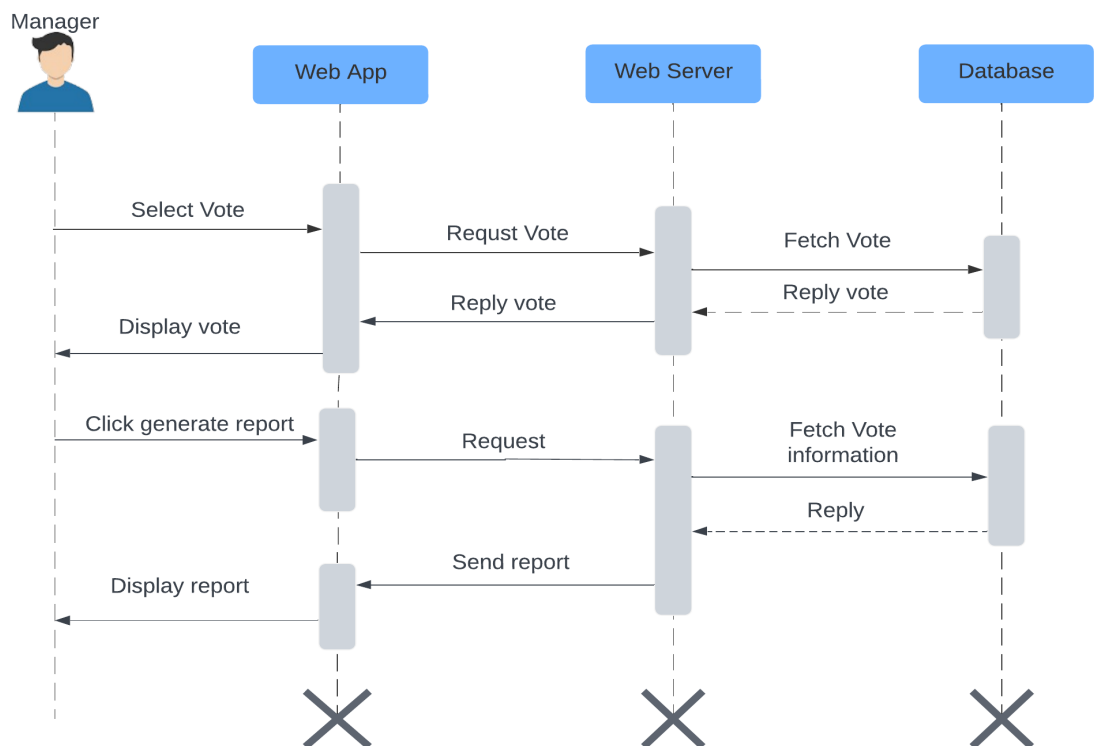


Figure 2.5: Sequence diagram for generating a report

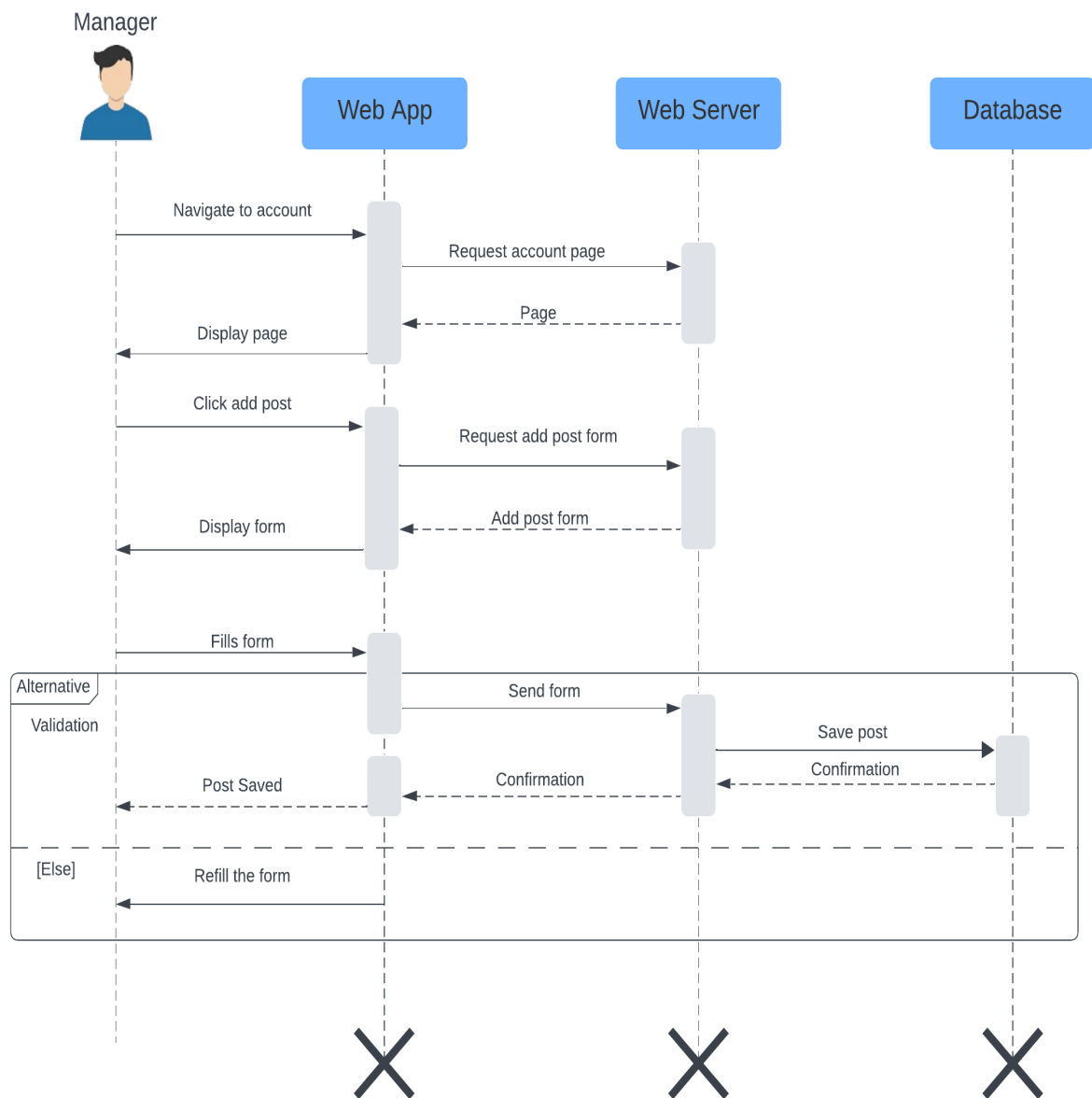


Figure 2.6: Sequence diagram for creating a post

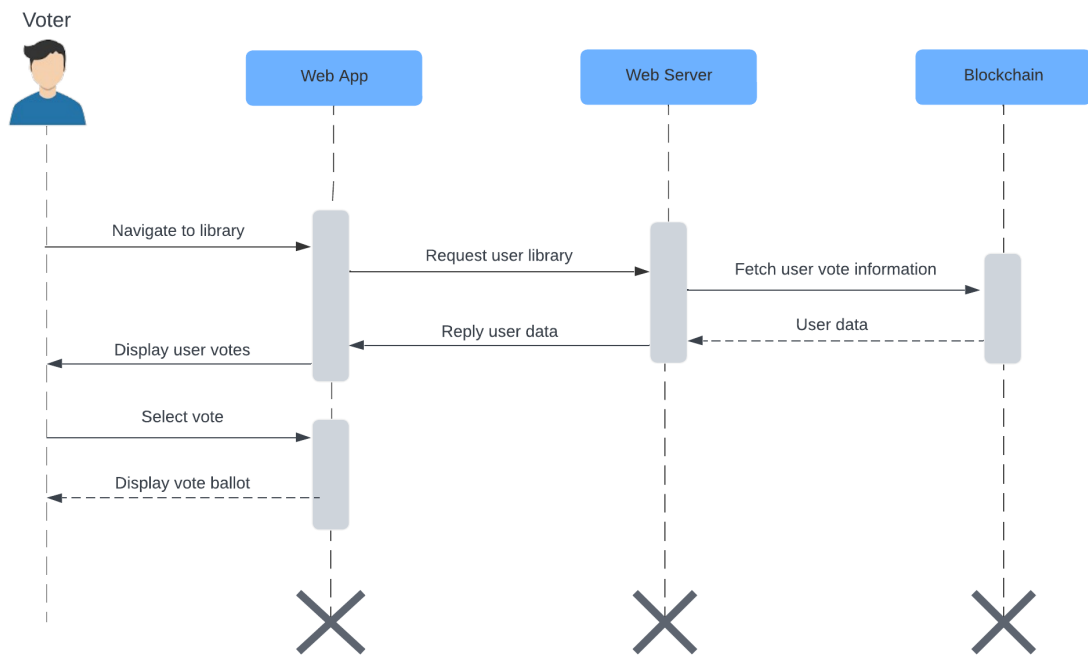


Figure 2.7: Sequence diagram for viewing voter history

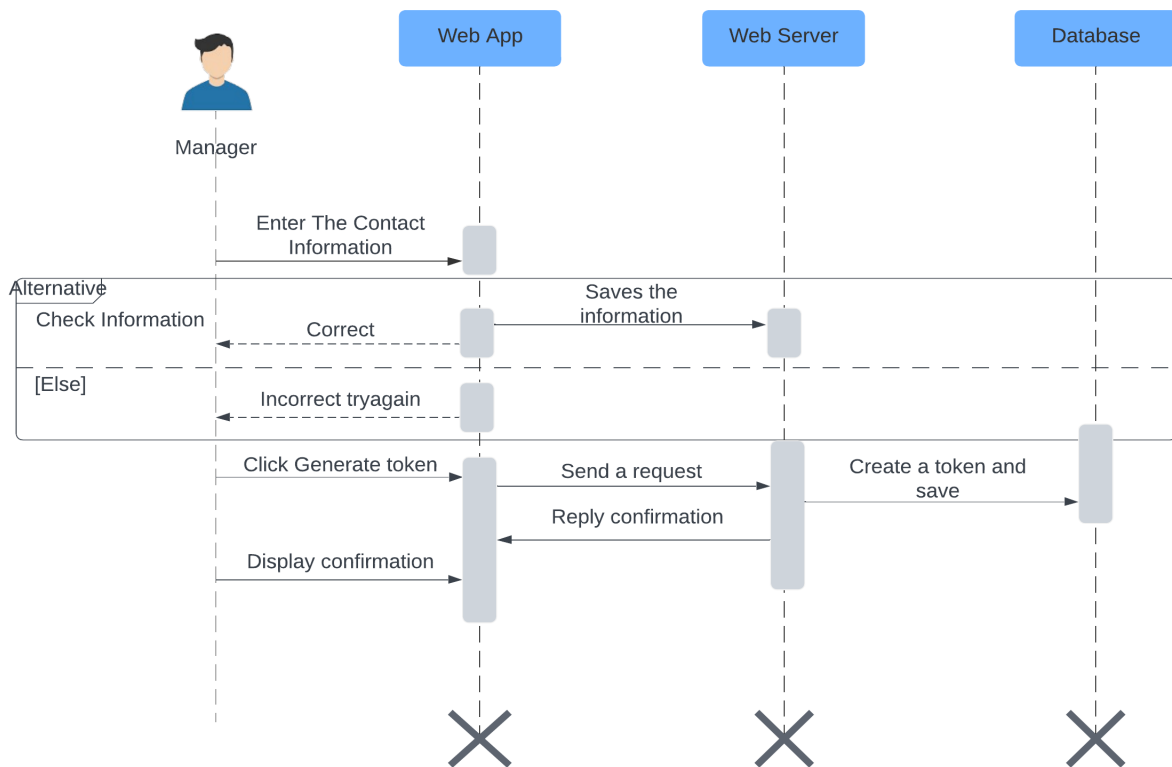


Figure 2.8: Sequence diagram for token generation

2.4.4.3 Activity Diagram

An activity diagram is a type of UML (Unified Modeling Language) diagram representing the workflow or the activities involved in a process. It visually depicts the sequence of activities, the flow of control, and the conditions for the transition from one activity to another [9]. We used activity diagrams to analyze the system.

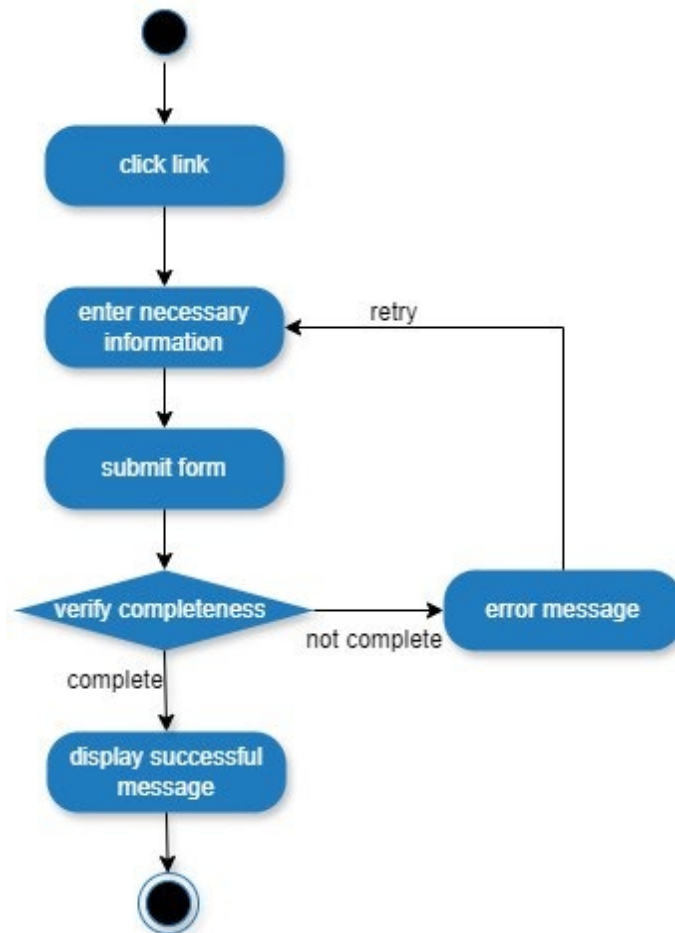


Figure 2.9: Activity diagram for creating an account using a unique token

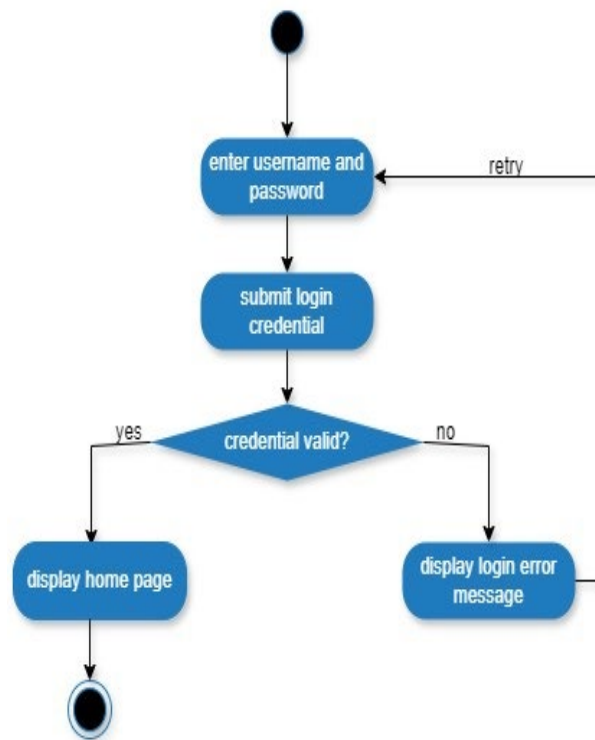


Figure 2.10: Activity diagram for login

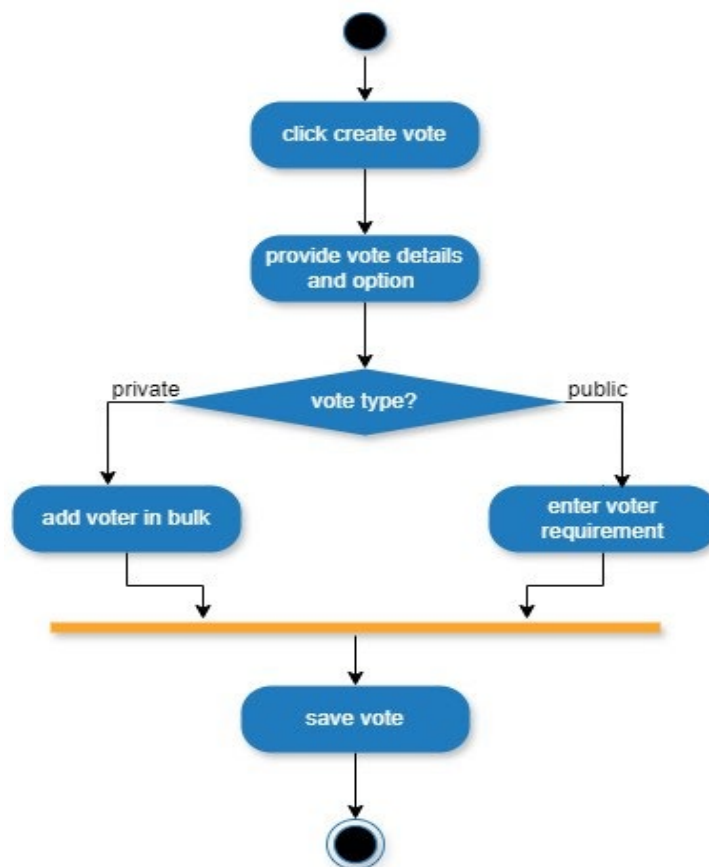


Figure 2.11: Activity diagram for creating a vote

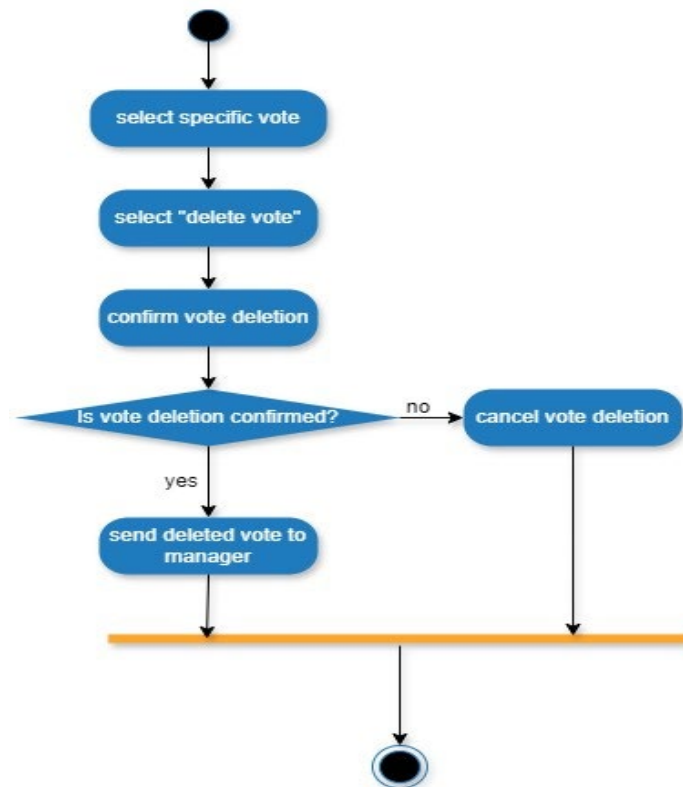


Figure 2.12: Activity diagram for delete vote

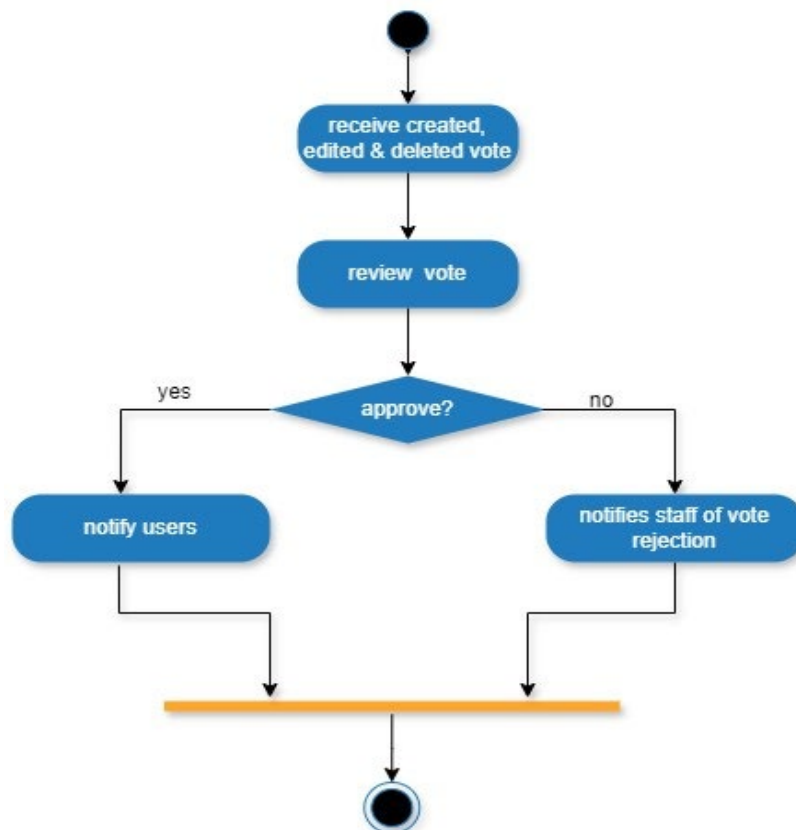


Figure 2.13: Activity diagram for vote approval

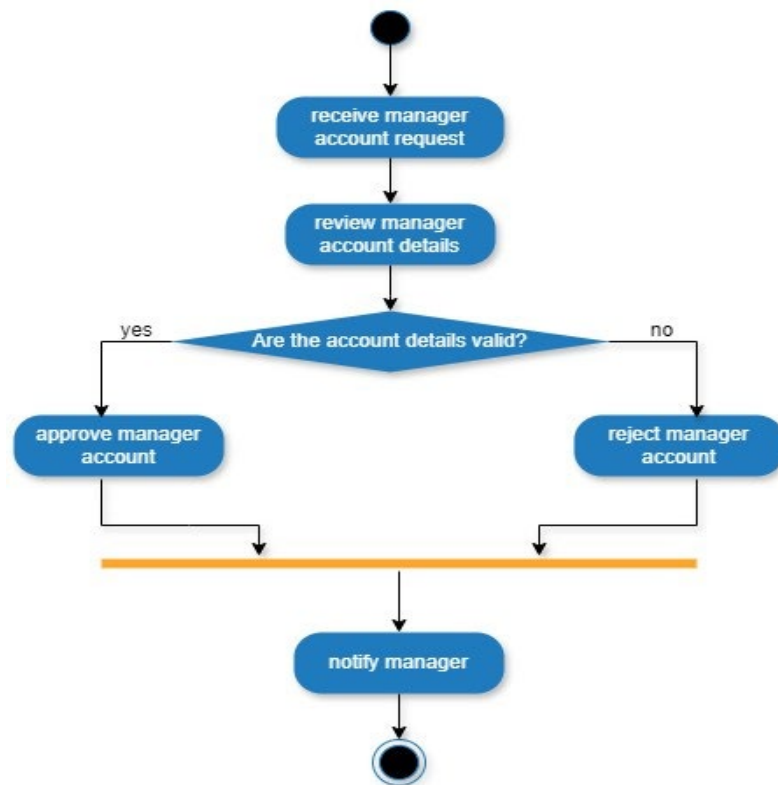


Figure 2.14: Activity diagram for the approval of manager account

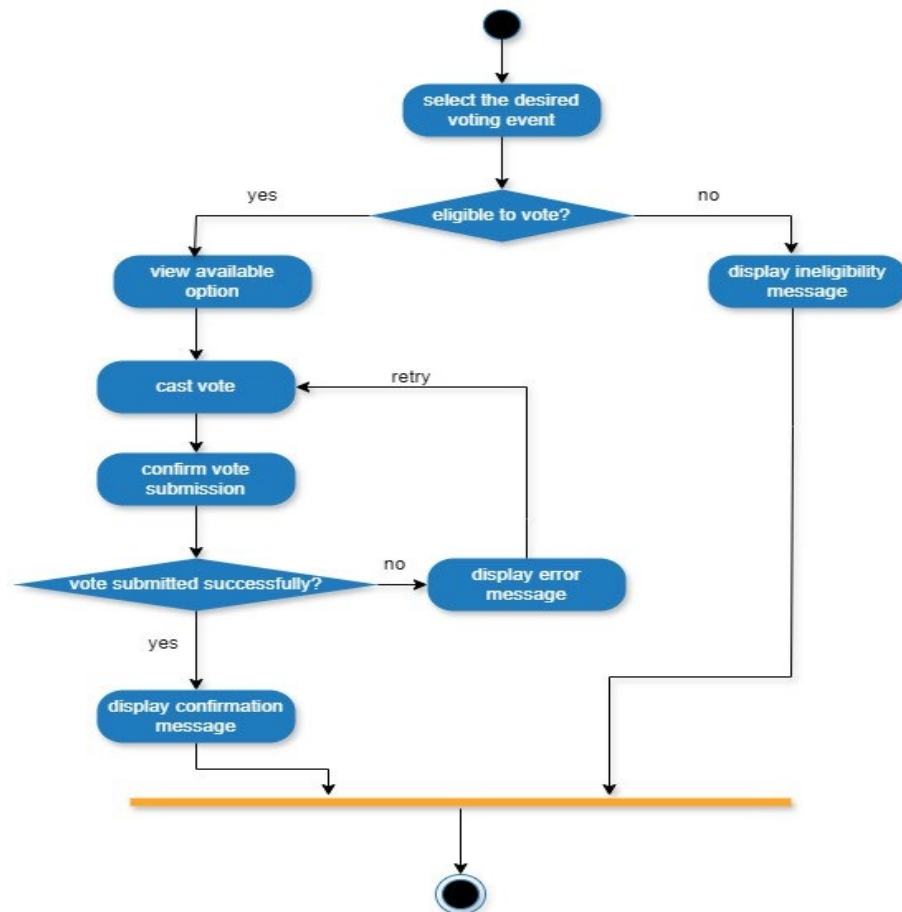


Figure 2.15: Activity diagram for cast vote

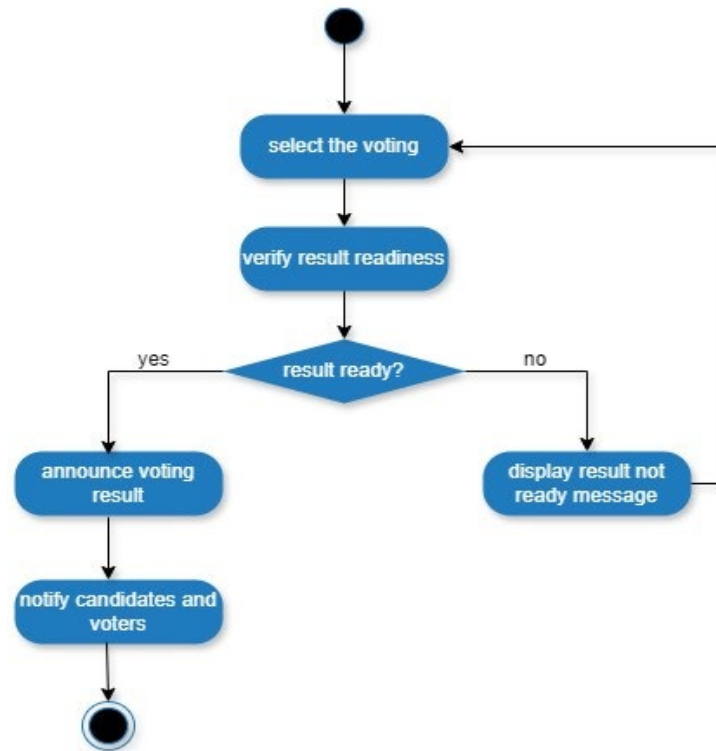


Figure 2.16: Activity diagram for announcing voting results

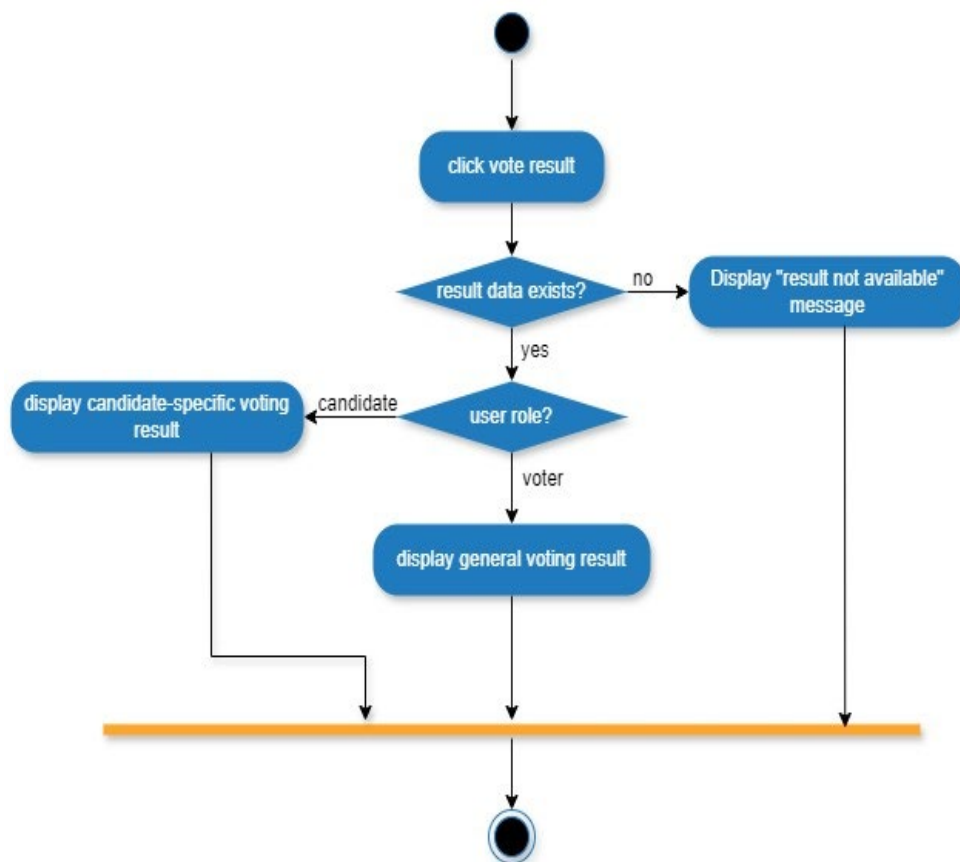


Figure 2.17: Activity diagram for viewing voting results

2.4.4.4 State Chart Diagram

A state chart diagram is a type of behavioral diagram that represents the behavior of a system or object over time. It shows the different states an object or system can be in and the events that cause it to transition between states. State chart diagrams help model the dynamic behavior of a system and understand how it responds to different inputs and events [10].

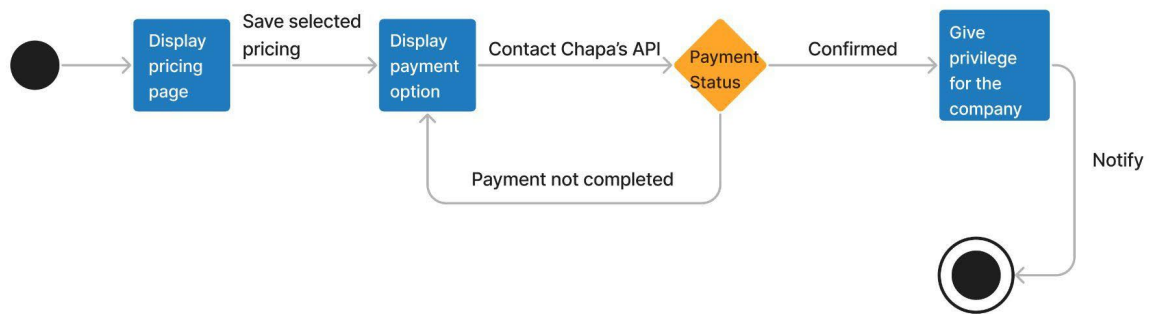


Figure 2.18: State chart diagram for making payment

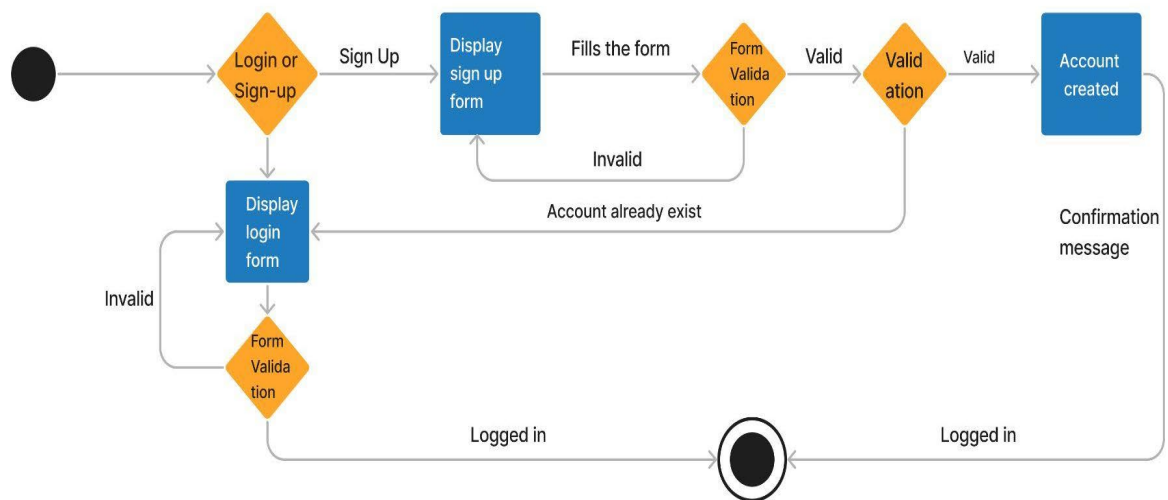


Figure 2.19: State chart diagram for signup and login

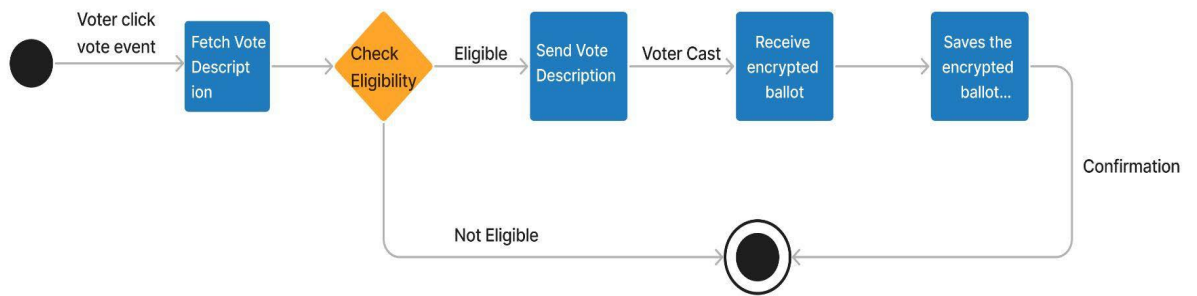


Figure 2.20: State chart diagram for casting vote

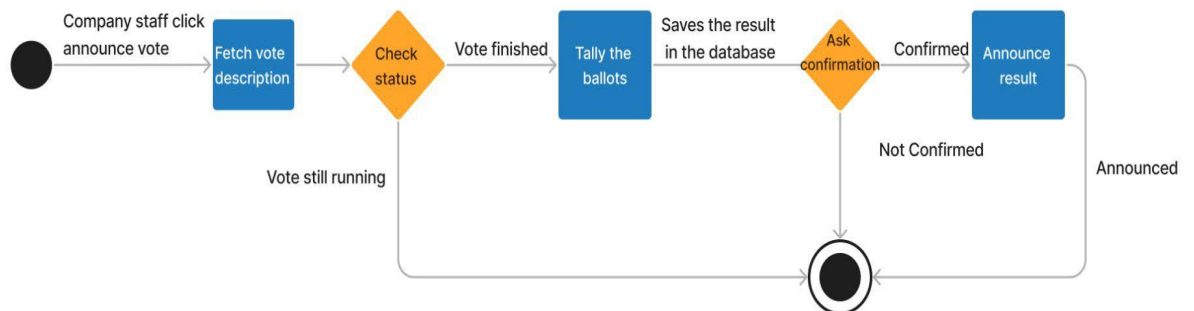


Figure 2.21: State chart diagram for announcing the voting result

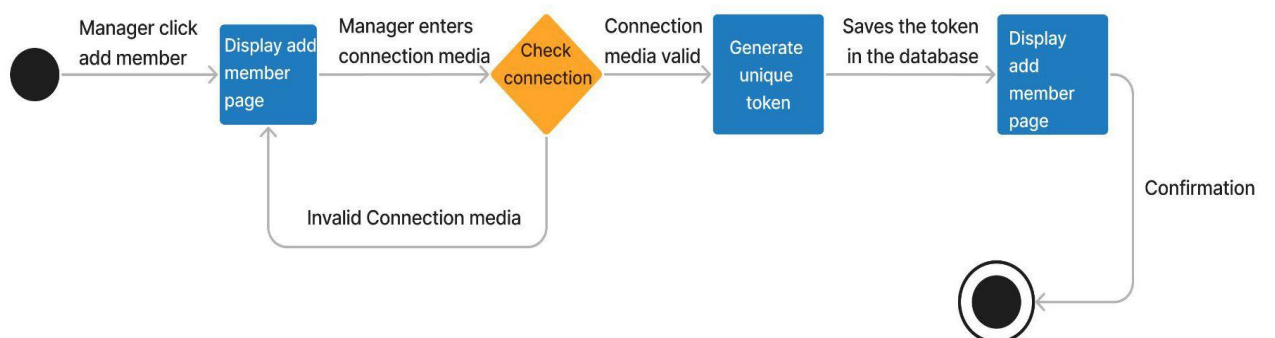


Figure 2.22: State chart diagram for generating a unique token

2.4.4.5 ER Diagram

An Entity-Relationship Diagram (ER Diagram) is a visual representation of the entities within a system and the relationships between those entities. It is used to illustrate the structure of a database clearly and systematically using squares, rhombus, and lines. ER Diagrams include entities, relationships, and attributes [11].

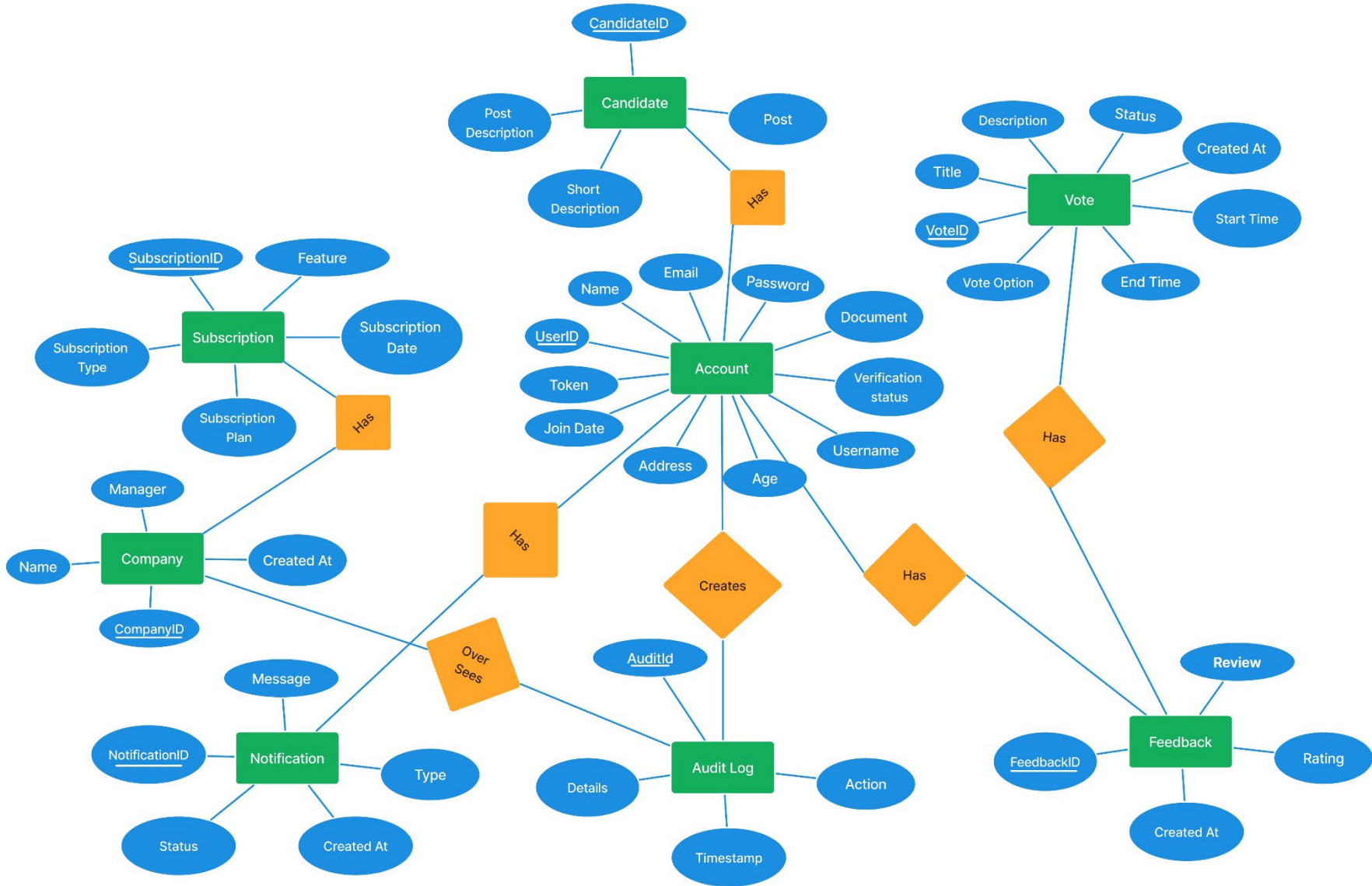


Figure 2.23: ER diagram

2.4.4.6 Class Diagram

A Class Diagram is a type of static structure diagram that describes the structure of a system by showing its classes, attributes, methods, and the relationships among objects. Class diagrams are essential in object-oriented design to visualize and document the different classes within a system and how they interact [12].



Figure 2.24: class diagram

2.4.4.7 User Interface Prototyping

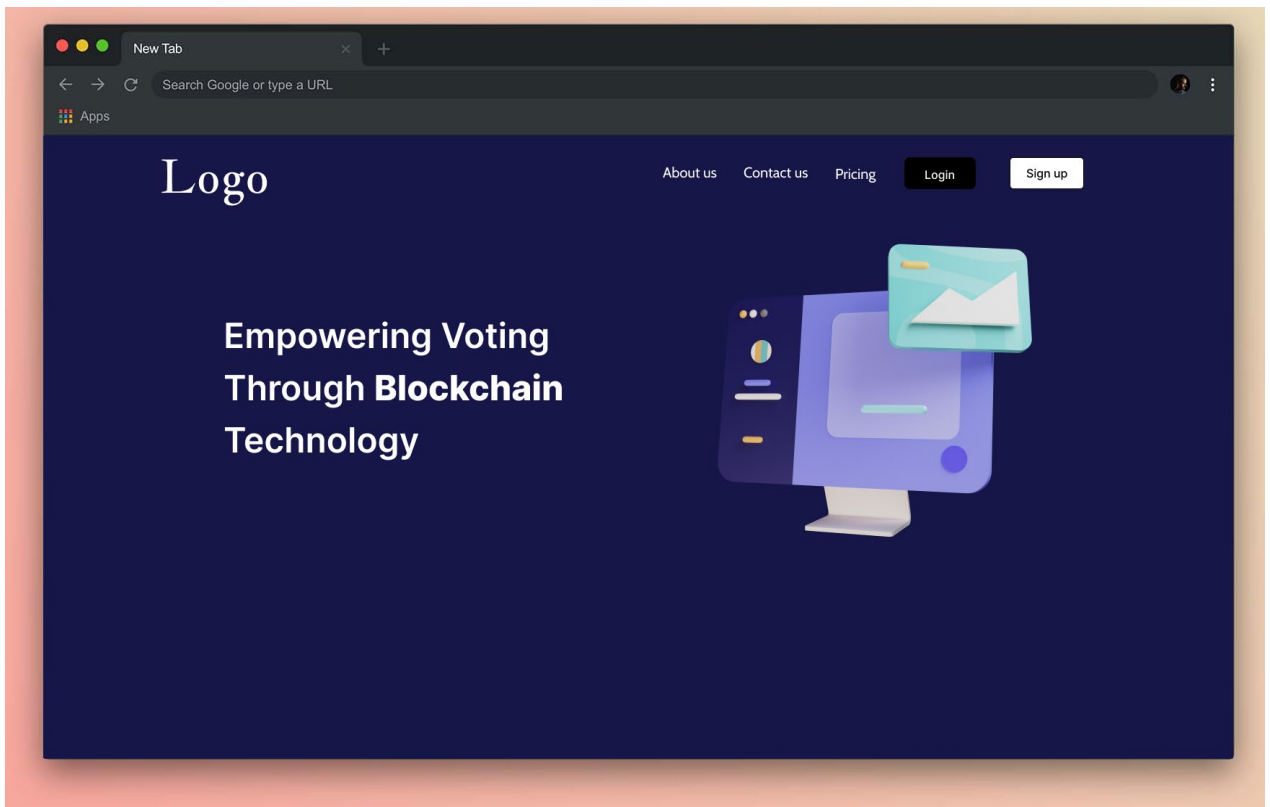


Figure 2.25: Home page

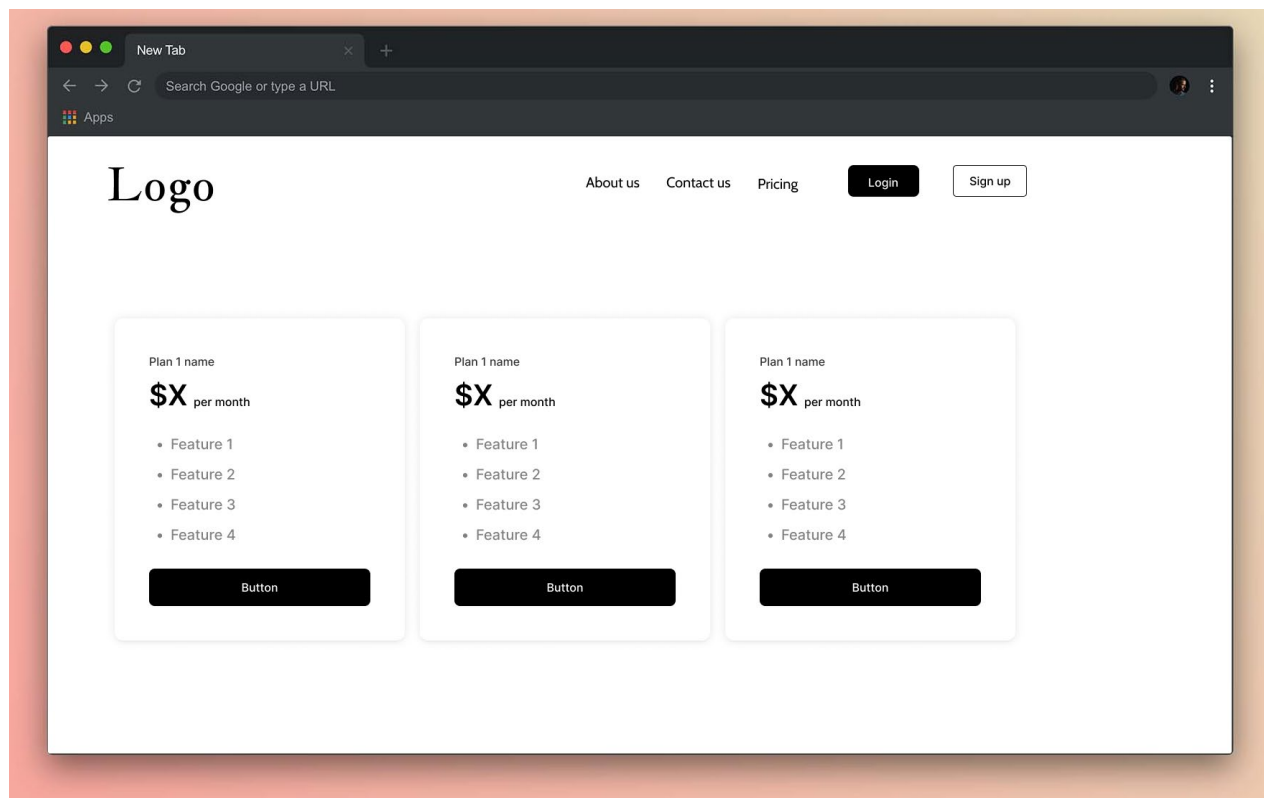


Figure 2.26: Pricing page

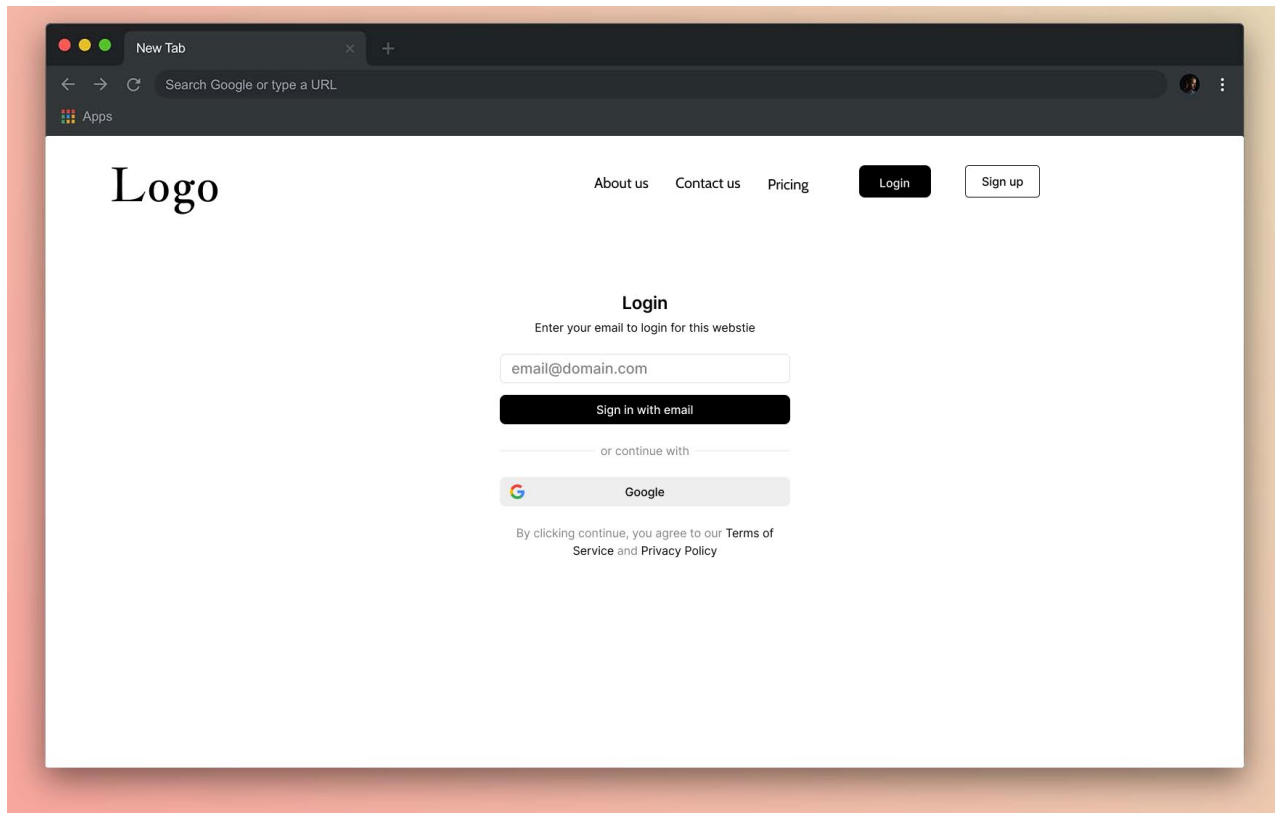


Figure 2.27: Login page

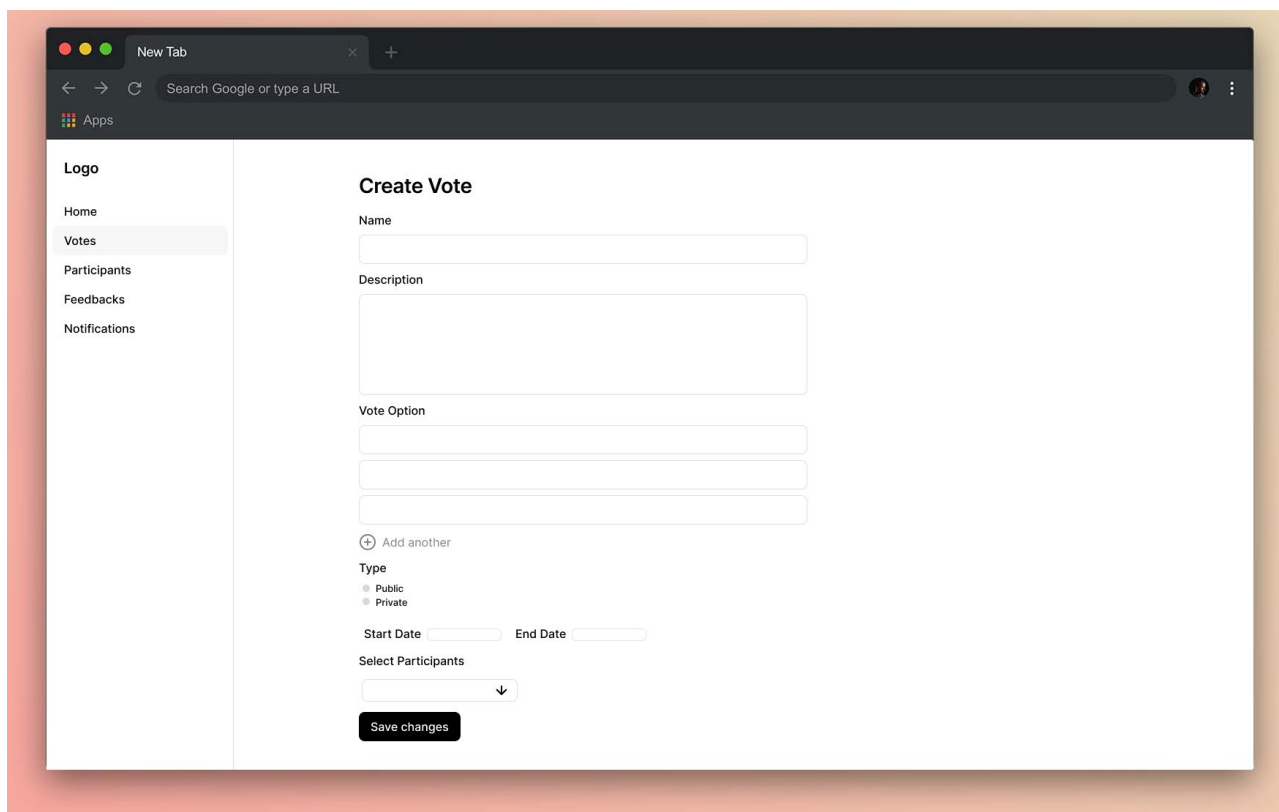


Figure 2.28: Vote creating page

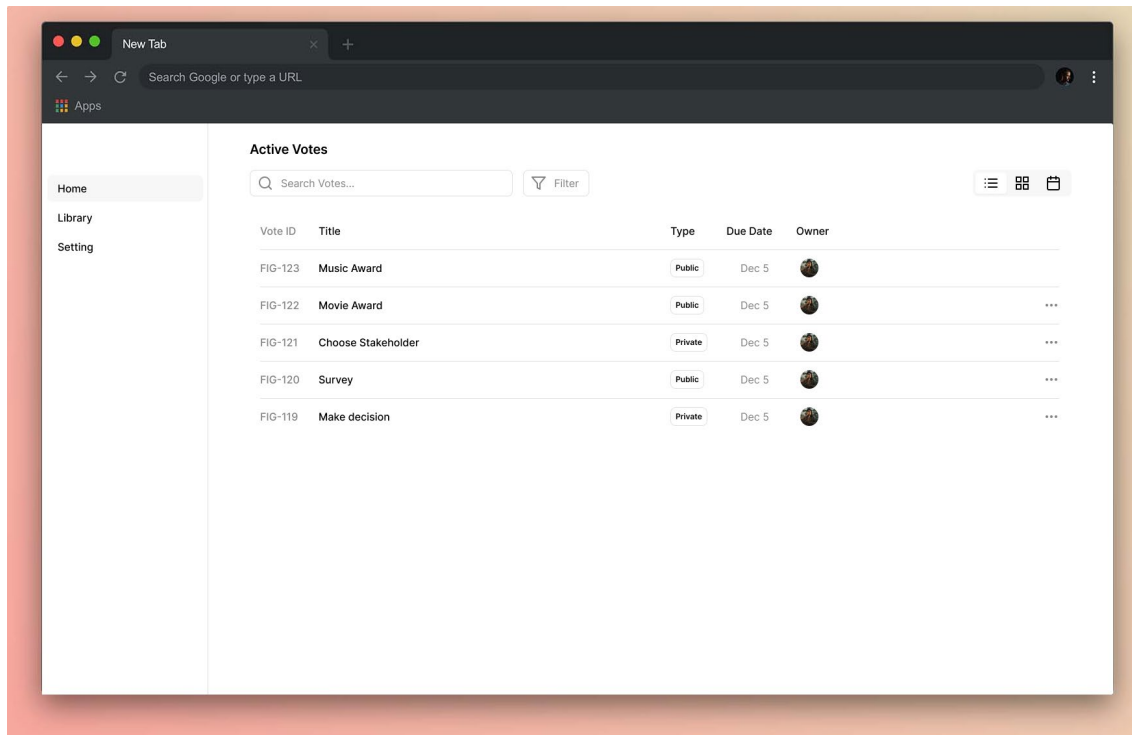


Figure 2.29: Voter home page

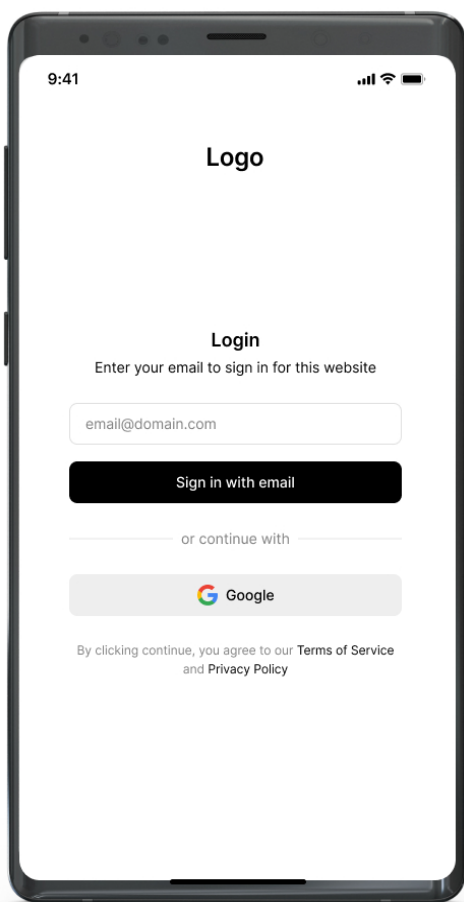


Figure 2.30: Login page layout on a phone

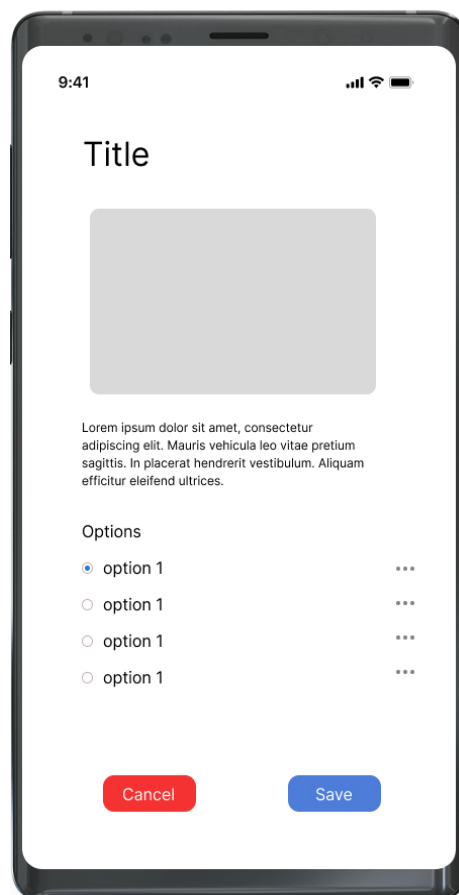


Figure 2.31: Vote casting on a phone

Chapter Three

System Design

3.1 Introduction

This chapter delves into the crucial system design concept, which is fundamental in creating a well-structured and efficient system that meets specific requirements and objectives. System design encompasses the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. Its primary goal is carefully planning and designing the system, ensuring it meets user needs and achieves the desired outcomes. This involves translating user requirements into a detailed blueprint that guides the implementation phase, considering scalability, reliability, security, and usability factors.

This chapter covers several elements for ensuring a robust and efficient system. We analyze the system's purpose and understand its objectives. Then, we propose a software architecture as a blueprint for arranging and interacting with components. Subsystem decomposition breaks down the system into manageable parts, aiding focused development and maintenance. The deployment diagram visualizes component distribution across hardware and software environments. Furthermore, detailed database design optimizes data storage, retrieval, and management. Lastly, we establish boundary conditions to define system behavior, including input handling, error management, and addressing limitations.

3.2 Purpose of the System

The purpose of the system is to provide a centralized and secure platform for managing various voting events. The system aims to achieve transparency, security, efficiency, and a consistent user experience using blockchain technology.

The system brings all voting events onto a single platform, enabling standardized processes, simplified administration, optimized resource allocation, and improved coordination between election authorities or organizations responsible for managing the voting processes. It is designed to be scalable and flexible, accommodating different scales of voting scenarios, from small-scale organizational voting to large-scale national elections.

Centralizing the voting system ensures a consistent user interface, registration process, and voting procedures across all types of voting. This centralization simplifies the voting experience, reduces confusion, and enhances user satisfaction.

Operating as a centralized system eliminates the need for separate infrastructure and systems for each event, resulting in cost savings, improved efficiency, and optimized resource allocation. It also enhances transparency and auditability by securely recording each vote as a transaction on the blockchain. This transparency fosters trust among participants and strengthens the credibility of the voting system.

The main purpose of integrating blockchain technology into our system is to secure the voting system and enhance its security and integrity. The decentralized nature of blockchain technology makes it difficult for any single entity to manipulate or tamper with votes. Through cryptographic algorithms and a consensus mechanism, the system ensures the integrity of the ballots, preventing fraudulent activities and ensuring accurate results.

Additionally, the system improves accessibility and convenience by providing an online platform. This allows voters to participate remotely, eliminating the need for physical presence at polling stations. It benefits individuals with mobility issues, remote voters, and those living in distant locations.

3.3 Design Goals

The design goals of this project are to achieve the following objectives:

- **Maximize System Usability:** One of the key design goals is to maximize system usability, aiming to create a user-friendly and intuitive interface for participants to navigate, register, and cast votes easily. By prioritizing usability, the voting system enhances the overall user experience, ensuring accessibility and ease of use for participants with varying levels of technical expertise.
- **Ensure Data Security:** Data security is paramount in a voting system. The design emphasizes data security by employing advanced techniques such as blockchain technology, cryptographic algorithms, access control mechanisms, encryption techniques, and regular security audits. These measures are implemented to protect the system against unauthorized access, data breaches, and potential cyber threats, ensuring the confidentiality and integrity of the voting data.
- **Optimize System Performance:** Optimizing system performance is a vital design goal to provide a smooth and efficient voting experience. This involves minimizing latency, handling high volumes of users, and optimizing resource allocation to ensure responsive operation, even during peak voting periods. By focusing on performance optimization, the voting system can deliver timely and efficient execution of voting processes.

- **Address Scalability:** The goal of addressing scalability is accommodating varying voting scales, from small-scale organizational voting to large-scale national elections. The system is designed to handle increased user loads and growing data volumes without compromising performance or security. Scalability considerations ensure that the system can adapt and scale effectively to meet the demands of different voting scenarios.
- **Ensure Reliability:** Reliability is a critical design goal in a voting system. The objective is to create a reliable system to withstand unexpected failures or disruptions. This is achieved by implementing redundancy, fault tolerance, and disaster recovery, ensuring continuous availability, and minimizing service interruptions during important voting events.
- **Enhance Maintainability:** The design goal of enhancing maintainability is to facilitate system updates, bug fixes, and future enhancements. This involves incorporating proper documentation, a modular code structure, and standardized development practices. The voting system can be easily maintained by prioritizing maintainability, enabling efficient updates, and long-term sustainability.

3.4 Current Software Architecture

As no current central voting system encompasses all voting processes, our development approach focuses on designing and implementing a new versatile software architecture that can cater to various voting systems. This architecture will be adaptable and modular, allowing for the integration of different components and functionalities based on specific voting requirements.

3.5 Proposed Software Architecture

We chose a hybrid architecture for the project software architecture. A hybrid architecture combining layers and microservices leverages the benefits of both architectural styles to build a robust, scalable, and maintainable system. Here's an in-depth look at these components and how they can meet various design goals.

Layered Architecture

1. Presentation Layer

This layer is responsible for the user interface and user interaction. It includes A web interface for users to interact with the system.

2. Application Layer

This layer handles routing requests from the presentation layer to the appropriate business logic or service. It includes an API gateway, which acts as the entry point for all client requests and routes them to the appropriate microservices.

3. Business Logic Layer

This layer contains the core functionality of the system, implemented as microservices. Each microservice is responsible for a specific domain of functionality.

Microservice Architectures

- User Service: Manages user accounts and authentication.
- Voting Service: Manages voting creation, modification, and tallying.
- Notification Service: Handles notifications to users.
- Report Service: Generates various reports.
- Subscription Service: Manages subscription plans for organizations and users.

4. Data Access Layer

This layer abstracts access to different data storage systems. It includes:

- Database Access: Manages interactions with relational and NoSQL databases.
- IPFS (Interplanetary File System) Access: Manages interactions with IPFS to store large files and documents.

5. Blockchain Layer

This layer ensures the immutability and transparency of voting data. It includes:

- Smart Contracts: Implements the voting logic on the blockchain.
- Blockchain Network: A decentralized network for validating and storing transactions.

6. Security Layer

This layer ensures the security of the entire system. It includes:

- Encryption: Ensures data is encrypted at rest and in transit.
- Authentication and Authorization: Manages user authentication and role-based access control.
- Audit and Compliance: Maintains logs and records for auditing purposes.

The hybrid architecture effectively meets various design goals by combining the structured approach of layered architecture with the flexibility and scalability of microservices.

Scalability is achieved by allowing each microservice to be independently scaled based on demand, such as increasing resources for the Voting Service during peak election periods. The layered architecture supports scaling specific layers, like the Presentation Layer, to handle more user requests.

Maintainability is enhanced as minor, independent microservices are more accessible to update and manage. Changes in one service do not affect others, and separating concerns across different layers simplifies system management.

Flexibility is ensured as each microservice can utilize different technologies and be developed by separate teams, allowing the best tools to be chosen for each task. The clear separation between layers facilitates technological changes without impacting the entire system.

Resilience is bolstered by the microservices' ability to isolate failures to individual services, allowing the rest of the system to continue functioning, and the layered architecture's ability to isolate issues in one layer from others.

Security is reinforced by each microservice's implementation of its security mechanisms, the API Gateway providing a centralized point for enforcing security policies, and the Security Layer ensuring comprehensive encryption, authentication, and auditing across the system. This hybrid architecture thus provides a robust, scalable, maintainable, flexible, resilient, and secure framework for a blockchain-based voting system [13].

3.5.1 Subsystem Decomposition

Subsystem decomposition involves breaking down the voting system into smaller, manageable subsystems. Each subsystem will focus on a specific aspect of the overall functionality. Here's a detailed breakdown of the subsystems of our project:

User Management Subsystem

The User Management Subsystem is fundamental for handling all user-related operations. It manages registering new users, verifies their identities, and creates user profiles. This subsystem also handles authentication, ensuring secure access through mechanisms like multi-factor authentication to add an extra layer of security. Profile management allows users to update their personal information, and role management assigns specific roles to users, such as admin or voter, dictating their permissions within the system. It depends on the vote management subsystem, voting process subsystem, and notification subsystem to handle votes and notifications.

Vote Management Subsystem

The Vote Management Subsystem is crucial for creating, updating, and deleting votes. It allows company staff to define the properties of each ballot, such as the title, description, and duration, and manage the list of eligible participants. This subsystem ensures that all vote-related data is correctly set up and worked throughout the voting process. It depends on the

candidate management, voting process, and blockchain integration subsystem to streamline the voting process.

Candidate Management Subsystem

The Candidate Management Subsystem focuses on handling candidates within the voting system. It allows the addition of new candidates, updating candidate details, and removing candidates as needed. This ensures that all candidate information is accurate and up to date, allowing voters to make informed decisions.

Voting Process Subsystem

The Voting Process Subsystem manages the core voting activities. This includes casting user votes and ensuring that each vote is securely recorded. It also handles counting votes, utilizing blockchain's inherent transparency and security features to prevent tampering. Once the voting period ends, this subsystem automatically announces the results, providing a transparent and verifiable outcome.

Notification Subsystem

The Notification Subsystem informs users by sending out notifications about the status and updates of votes. It creates and delivers notifications through various channels such as email, SMS, or in-app alerts, ensuring users are continually updated on important events and changes. It depends on the vote management and process subsystem to handle the notification.

Blockchain Integration Subsystem

The Blockchain Integration Subsystem interfaces with the blockchain network, which is the backbone of the voting system. It manages the deployment and execution of smart contracts, self-executing contracts with the terms directly written into code. These smart contracts handle vote casting, user verification, and result announcement, ensuring that all transactions are secure, transparent, and immutable. This subsystem also manages blockchain transactions, ensuring they are processed and recorded correctly. Integrate with the vote management and storage subsystem to securely manage the vote and store the vote data.

Storage Subsystem

The Storage Subsystem manages the storage of voting data and related files. It leverages decentralized storage solutions like the Interplanetary File System (IPFS), a distributed file system that aims to connect all computing devices to the same file system and allows users to host and share content in a decentralized manner. This ensures data integrity and availability. Additionally, it manages traditional database systems to store user information and other non-blockchain data, providing a comprehensive and reliable data storage solution.

Audit and Monitoring Subsystem

The Audit and Monitoring Subsystem is essential for maintaining transparency and accountability. It monitors real-time system performance, ensuring issues are detected and addressed promptly. This subsystem also logs all actions for audit purposes, creating an immutable record of all system activities. Data analytics tools within this subsystem are analyzed.

Subscription management subsystem

The subscription management subsystem in a blockchain-based voting system handles all aspects of user and organization subscriptions to ensure valid, active participation in voting. It includes managing subscription plans, organizational subscriptions, payment processing, and notifications. Subscription plans can be created, updated, and deleted as needed. Organization subscriptions are managed to facilitate subscribing, renewing, and canceling. Payment processing ensures secure transactions, including invoicing and tracking. Notifications inform organizations about their subscription status, renewals, and expirations. This subsystem provides a seamless, scalable, and efficient process for managing subscriptions.

3.5.2 Component Diagram

A component diagram is a type of UML (Unified Modeling Language) diagram that illustrates the organization and interrelationships of the components within a system. Components represent modular parts of the system that encapsulate behavior and data, making it easier to manage complex software architectures [14].

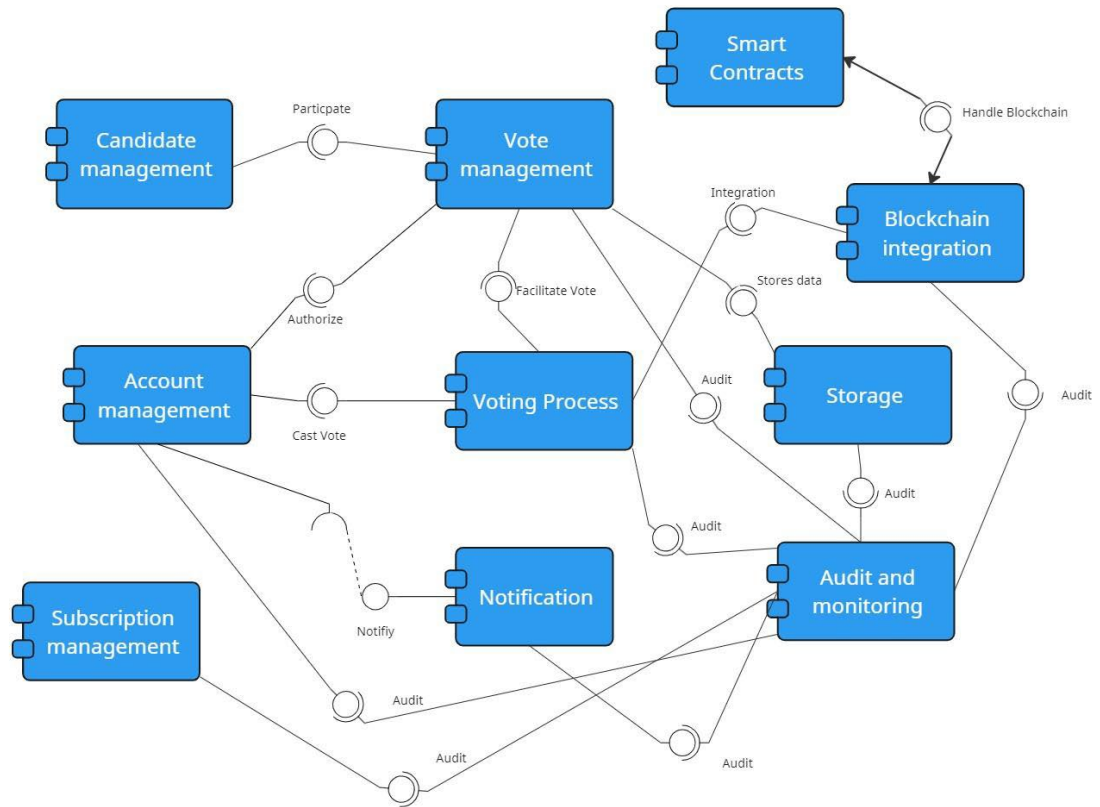


Figure 3.1: Component diagram

3.5.3 Deployment Diagram

A deployment diagram is a type of UML (Unified Modeling Language) diagram that shows the physical deployment of artifacts (software components, hardware nodes, etc.) on nodes (hardware devices, execution environments). It depicts a system's hardware and software configuration, showing how the software components are deployed on the hardware infrastructure [15].

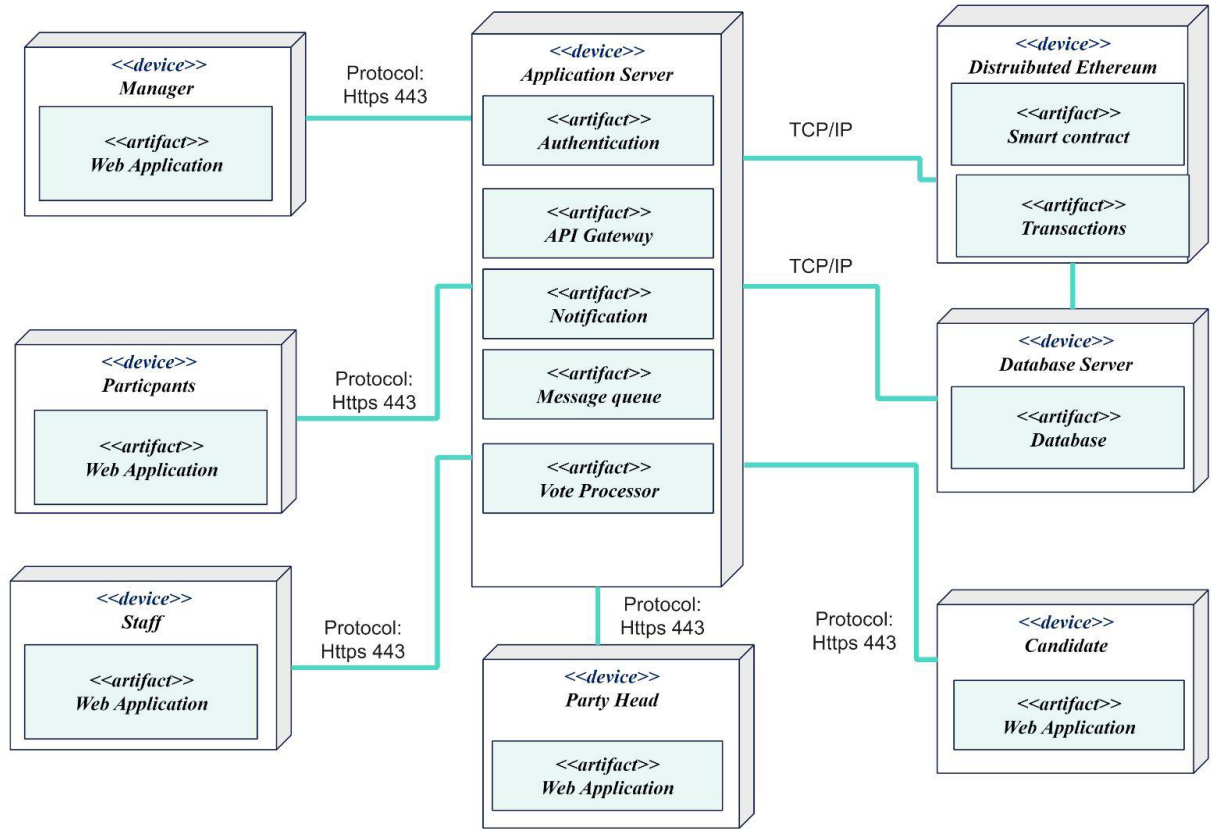


Figure 3.2: Deployment diagram

3.5.4 Persistent Data Management

The proposed voting system will manage various data types, including documents, audio, video, and user information. The system will organize documents and store user information, allowing candidates to upload campaign videos and audio.

For the project, the chosen database technologies are PostgreSQL and MongoDB. PostgreSQL is an open-source relational database management with reliability, robustness, and extensibility. It provides a structured and organized approach to storing and managing data, making it suitable for storing user information, voting records, and other structured data. PostgreSQL offers advanced features such as solid data integrity constraints, transaction support, and support for complex queries, essential for maintaining data accuracy and consistency.

MongoDB, on the other hand, is a popular NoSQL database that stores data in flexible, JSON-like documents. This makes it well-suited for storing unstructured or semi-structured data, including documents, audio, video, and other media files. MongoDB's document-oriented approach allows easy scalability, as data can be stored and retrieved without

complex relational schemas. It also provides high availability and fault tolerance, ensuring data accessibility even during server failures.

PostgreSQL and MongoDB offer data validation rules, constraints, and transactions to ensure data integrity. These features help maintain the consistency and accuracy of the stored data. Additionally, PostgreSQL's ACID (Atomicity, Consistency, Isolation, Durability) compliance guarantees that database transactions are executed reliably and that the data remains consistent.

Data access methods will include SQL queries for structured data manipulation in PostgreSQL, allowing efficient retrieval and modification of user information and voting records. MongoDB's flexible document model will facilitate easy querying and retrieval of unstructured or semi-structured data, such as documents, audio, and video files.

Considering data backup and recovery, AWS will deploy web applications, databases, and test servers. AWS provides robust backup and recovery mechanisms, including automated snapshots, point-in-time recovery, and replication options for PostgreSQL and MongoDB. These features ensure data durability and allow for timely recovery in case of data loss or system failures.

3.5.5 Detailed Database Design

3.5.5.1 Relational Table

Account table	
PK->UserID	Int
Full Name	Varchar(255)
Email	Varchar(255)
Password	Varchar(255)
Address	Varchar(255)
Age	Int
Join Date	DateTime
Token	Varchar(255)
Username	Varchar(255)
Verification Status	Bit
Verification Document	Varchar(255)

Table 3.1: Account table

Vote Table	
PK->VoteID	Int
Title	Varchar(255)
Description	Varchar(255)
Status	Enum
Created At	DateTime
Start Time	DateTime
End Time	DateTime
Vote Option	Varchar(255)
FK->participants	Int

Table 3.2: Vote table

Company Table	
PK->CompanyID	Int
Name	Varchar(255)
Manager	Varchar(255)
Created At	DateTime

Table 3.3: Company table

Candidate Table	
PK->candidateID	Int
FK->UserID	Int
Post	Varchar(255)
Short Description	Varchar(255)
Post Description	Varchar(255)

Table 3.4: Candidate table

Feedback Table	
PK->FeedbackID	Int
FK->VoteID	Int
FK->UserID	Int
Review	Varchar(255)
Rating	SmallInt
Created At	DateTime

Table 3.5: Feedback table

Notification Table	
PK->NotificationID	Int
FK->UserID	Int
Message	Varchar(255)
Type	Enum
Created At	DateTime
Status	Enum

Table 3.6: Notification table

Audit Log table	
PK->AuditID	Int
Fk->UserID	Int
Fk->companyID	Int
Action	Enum
Timestamp	DateTime
Details	Varchar(255)

Table 3.7: Audit log table

Subscription table	
PK->SubscriptionID	Int
Fk->companyID	Int
SubscriptionType	Enum
SubscriptionPlan	Varchar(255)
Feature	Varchar(255)
SubscriptionDate	DateTime

Table 3.8: Subscription table

3.5.5.2 Database Normalization

Database normalization is organizing data to reduce redundancy and improve data integrity. This is achieved by structuring the data into tables and establishing relationships between them according to rules (standard forms) to ensure consistency and eliminate anomalies in data operations.

In this project, we will use three database normalization forms: first normalization, second normalization, and third normalization.

Let's normalize each table one by one:

Account table:

This table contains non-atomic values because the attribute verification document can have multiple files. We will create a separate table to hold verification documents and tokens to fix this.

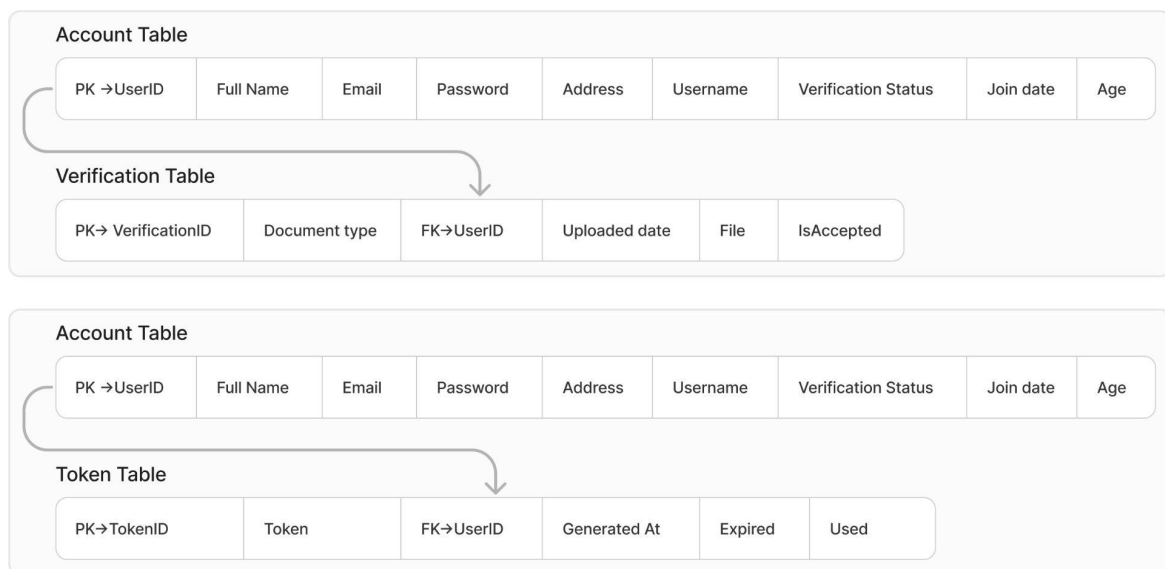


Table 3.9: Account table normalization

The non-key attributes only depend on the key characteristics, so the table is in the 2nd normalized form. Also, there is no transitive dependency among attributes.

Name, Address, and Created At are directly dependent on CompanyID. Still, a manager might represent a transitive dependency, as it might be more closely associated with the concept of a person rather than directly with the company itself.

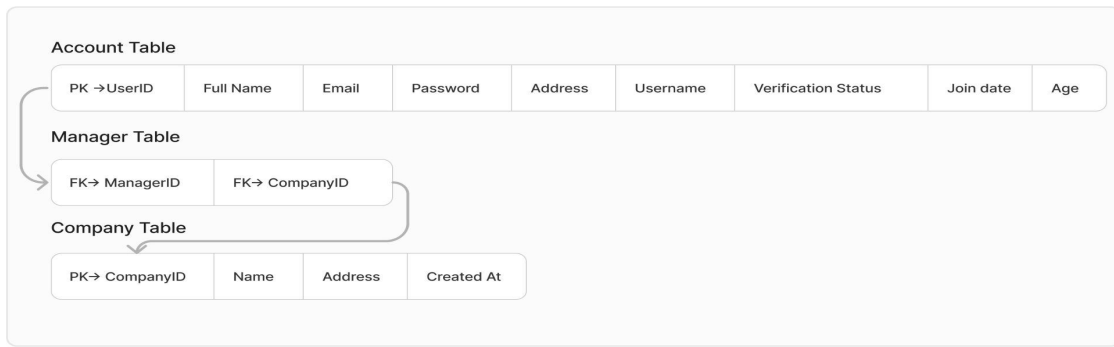


Table 3.10: Company table normalization

Vote Table:

We must ensure that each cell contains atomic values and no repeating groups. We will create their table since the vote option and participants attribute are not atomic values.

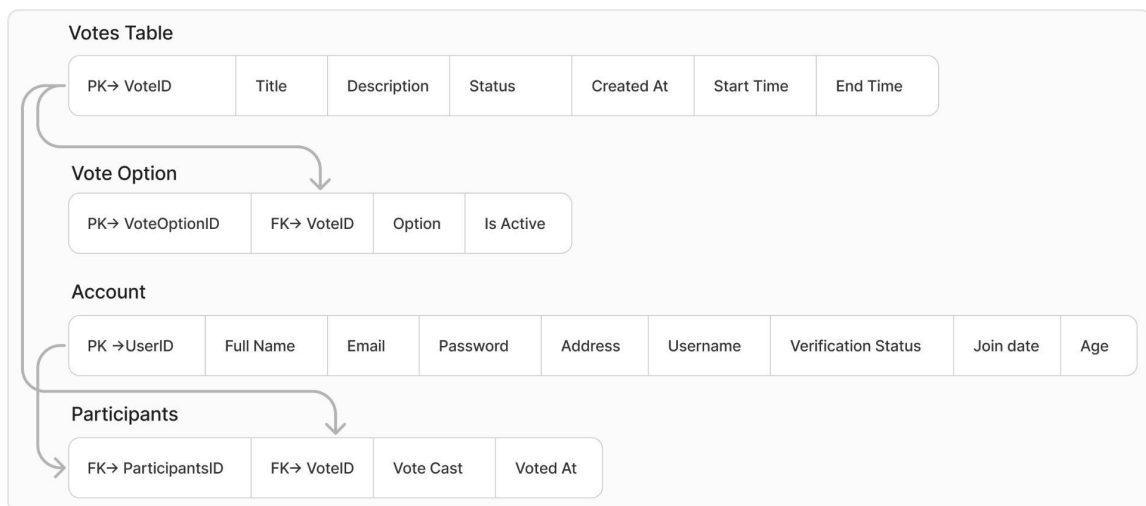


Table 3.11: Vote table normalization

In the Votes table, all non-key attributes (Title, Description, Status, Created At, Start Time, End Time) are fully functionally dependent on the primary key (VoteID), So the table is already in the Second Normal Form (2NF), also in third standard form (3NF)

Candidate table:

There's a partial dependency since the attribute Post depends only on part of the primary key (CandidateID). To address this, we'll create a new table for Post.

Now, the Candidate table will contain attributes entirely dependent on the primary key (CandidateID), and the Post table will contain posts associated with each candidate.

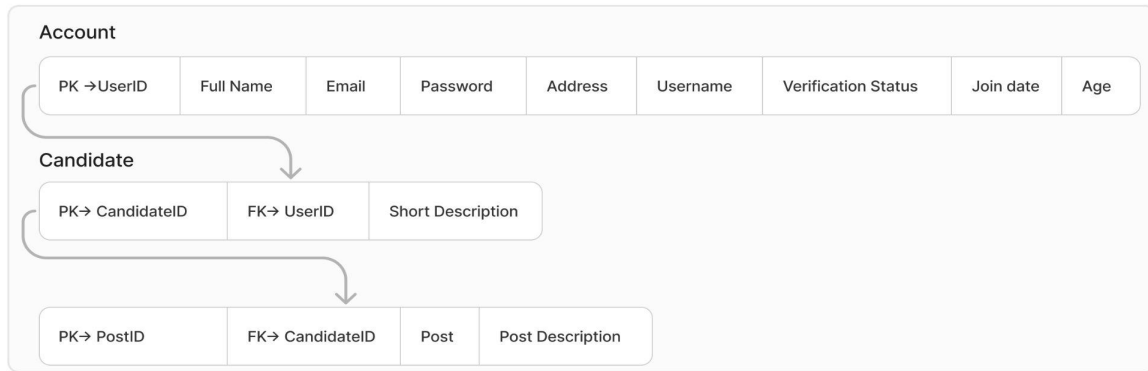


Table 3.12: Candidate table normalization

The table is already free from transitive dependencies, as each non-key attribute depends directly on the primary key (CandidateID).

Subscription table:

This table has many non-atomic values, so to normalize this table, we will create a separate table for subscription plans and features.

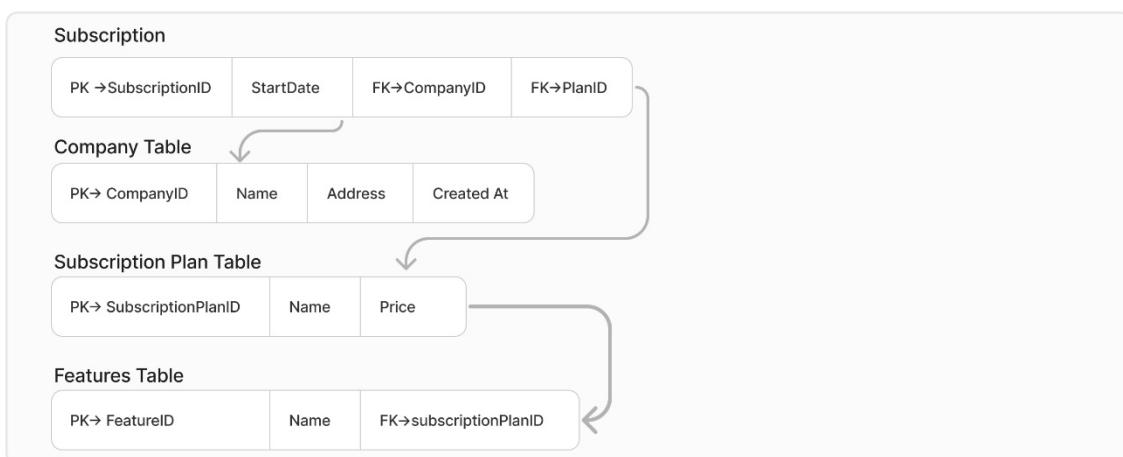


Table 3.13 Subscription table normalization

The table is already in 2NF and 3NF form because there is no partial dependency between non-key attributes and no transitive dependency.

3.5.5.3 Extended Entity Relation Diagram

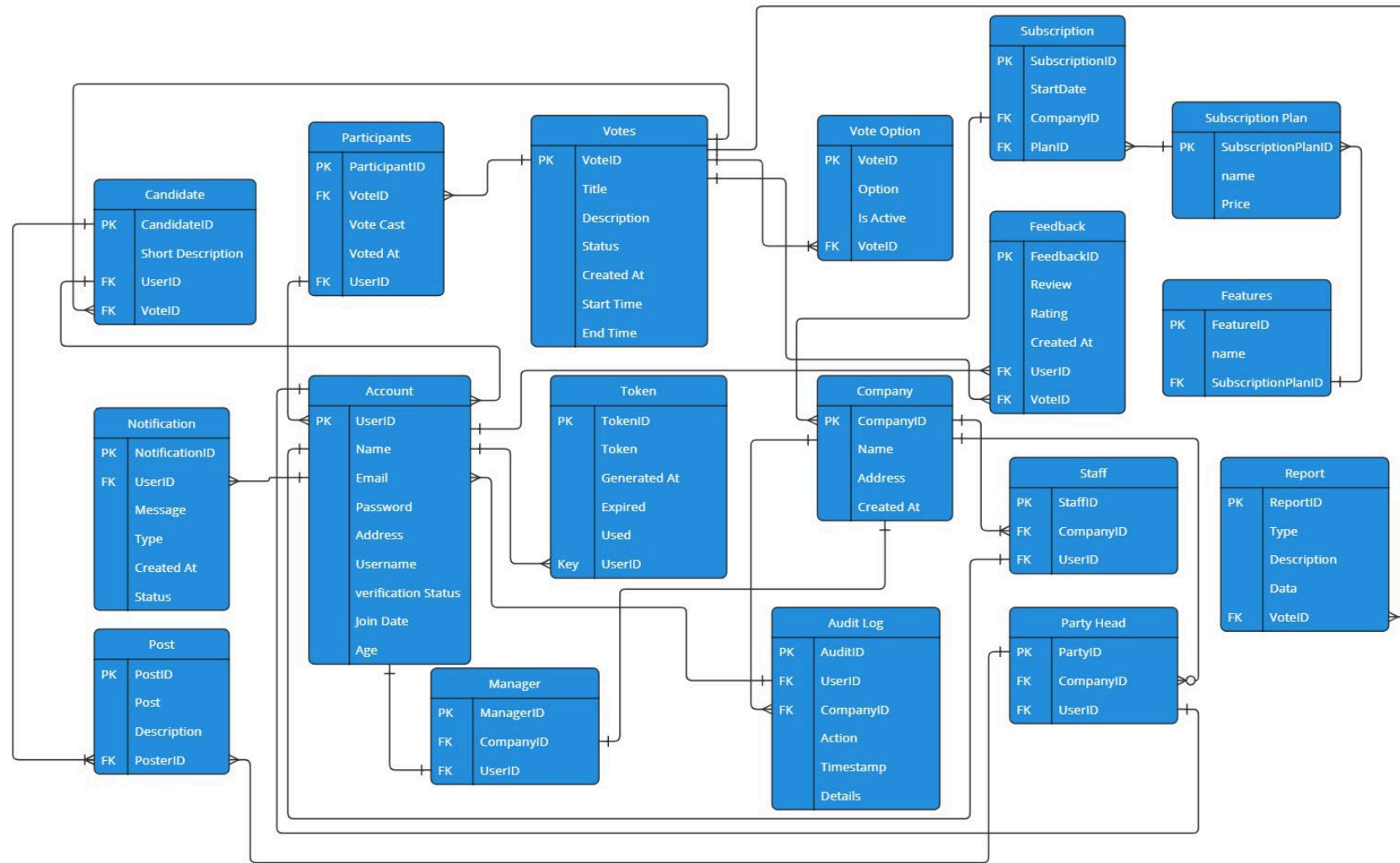


Figure 3.3: Normalized ER diagram

3.5.5.4 OO-Relational Mapping

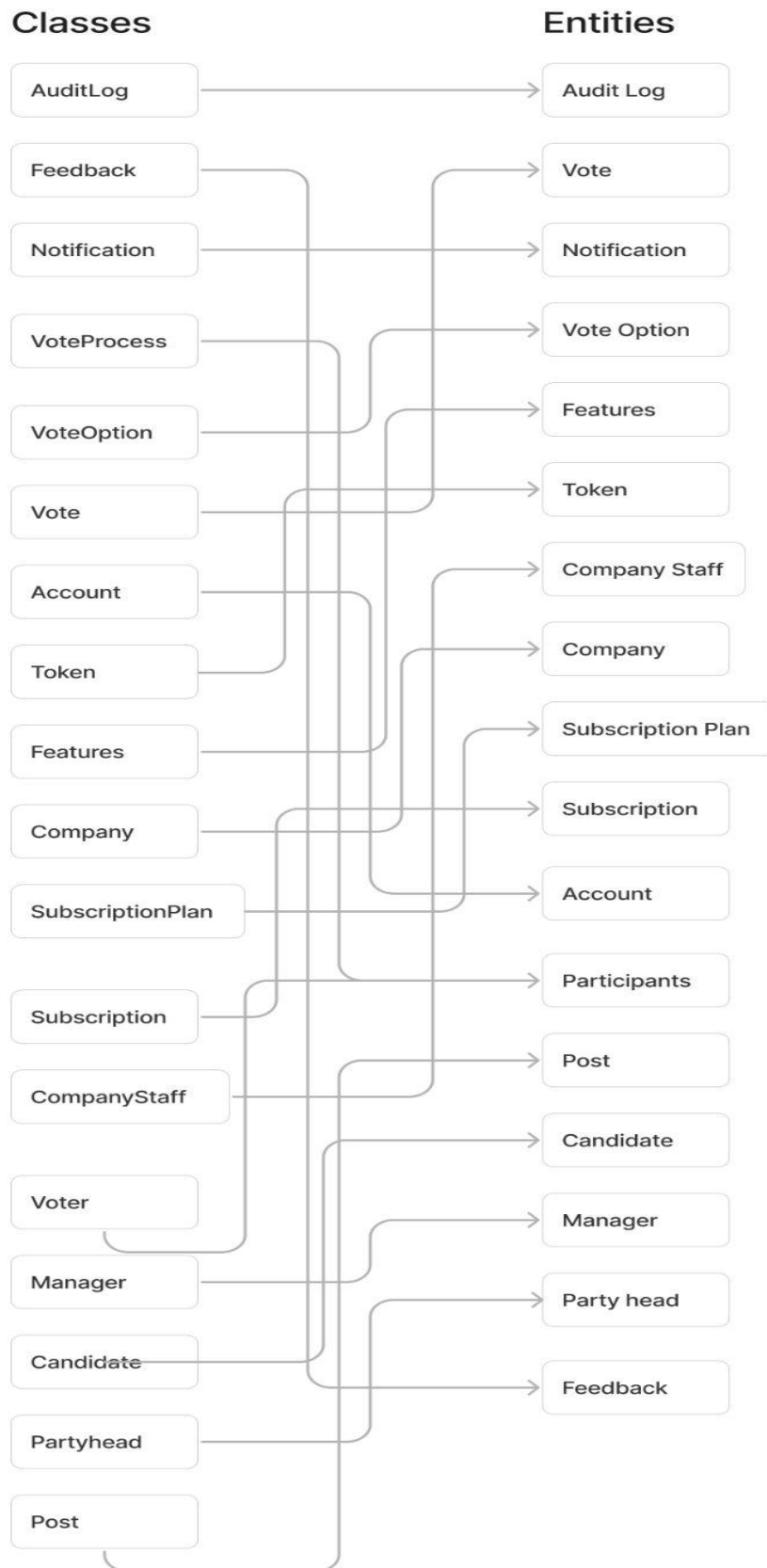


Figure 3.4: OO-relational mapping

3.5.6 Access Control and Security

This voting system's Access Control and Security aspect focuses on implementing robust measures to ensure its security and control access to its resources. This includes safeguarding data privacy and security, enforcing user authentication and authorization, and applying data encryption methods.

The system implements strict measures to maintain data privacy and security. Users are required to create accounts by providing necessary identification information, which is securely stored and protected. User authentication is enforced, ensuring that only authorized individuals can access their respective accounts. This authentication process may involve using unique login credentials, such as usernames and passwords, or more advanced methods like two-factor authentication. By implementing strong authentication mechanisms, the system effectively prevents unauthorized access and protects against identity fraud.

Authorization plays a critical role in controlling user access to different system resources. Depending on their roles and responsibilities, participants in the voting system, such as voters, managers, staff members, candidates, and party heads, are granted specific permissions and privileges. This approach ensures that each authorized user can perform actions aligned with their respective roles, such as casting votes, managing voter registration, or accessing sensitive data.

To facilitate the account creation process, a unique token is generated for each user, including candidates, staff members, and party heads, who hold specific roles in the voting system. The manager responsible for granting access to users, generates these tokens, which act as digital credentials and are distributed exclusively to the respective individuals. By distributing unique tokens, the system guarantees that only authorized users with specific roles can create accounts and participate in voting.

Blockchain technology ensures the confidentiality, integrity, and security of data in the voting system. Sensitive information is encrypted within the blockchain, ensuring it remains unreadable without decryption keys. The blockchain acts as a decentralized and tamper-resistant ledger, recording all transactions and activities. It guarantees the immutability and transparency of the voting process, reducing the risk of manipulation or unauthorized access. By using encryption within the blockchain, the system effectively protects sensitive data, maintains transaction integrity, and builds trust among participants.

By integrating these access control and security measures, the voting system aims to create a secure environment where data privacy is maintained, user authentication is enforced, proper

authorization is implemented, and data encryption methods are applied to protect sensitive information. These measures collectively contribute to the overall security and integrity of the voting system, instilling confidence in participants and maintaining the trust of stakeholders.

3.5.7 Global Software Control

The global software control in the voting system ensures the processing of requests, coordination of workflows, and generation of appropriate responses. Specifically, it manages the following components:

1. Request Initiation:

- **User Account Creation:** When a user initiates a request to create an account by providing identification information, the system receives and verifies the data. If the provided information is valid, the system creates a user profile.
- **User Login and Authentication:** When a user initiates a request to log in to their account using their credentials, the system performs controlled access and authentication processes to validate their login credentials.
- **Account Management:** When users request to update their personal information or change passwords, the system allows them to modify their account details accordingly.
- **Payment Management:** When a manager initiates a request to make payments for system-related expenses, the system processes the payment request and manages the payment transaction accordingly.
- **Token Generation for Account Creation:** When a manager initiates a request to generate unique tokens for staff members, parties, or candidates, the system generates the tokens as requested.

2. Workflow Coordination:

- **User Workflow:** The system coordinates user-related workflows, including account creation, profile updates, and password changes. It ensures the necessary steps and validations are followed to maintain data integrity and security.
- **Voting Workflow:** The system facilitates the coordination of the voting workflow. It ensures the creation of voting events, participant eligibility checks, secure vote casting, and result generation. The system provides a logical and controlled progression of the workflow.

- **Manager Workflow:** The system handles manager-related workflows, including initiation and approval/rejection of manager account creation requests. It performs necessary checks and validations before granting managerial access.
- **Notification Workflow:** The system coordinates the generating and delivering of notifications to users. It determines appropriate message channels and ensures timely updates on vote status and essential events.

3. Synchronization:

Synchronization is crucial to avoid conflicts and maintain data consistency. In this project, the following synchronization issues can arise:

- **Concurrent Access to Shared Resources:** Multiple subsystems may need to access shared resources simultaneously. To address this, synchronization mechanisms like locks can be implemented to ensure exclusive access and prevent data corruption.
- **Data Consistency:** Maintaining data consistency becomes vital when multiple subsystems are processing requests. Transaction management techniques, such as ACID (Atomicity, Consistency, Isolation, Durability), can ensure that data remains consistent throughout the system.
- **Race Conditions:** Race conditions may occur when multiple subsystems or threads attempt to access and modify shared data simultaneously. Proper synchronization techniques, such as critical sections or synchronized methods, can be implemented to prevent race conditions and ensure predictable behavior.

To address synchronization issues, the following measures can be implemented:

- **Designing a robust data access and modification strategy:** Implementing well-defined protocols for accessing and modifying shared data helps prevent conflicts and ensures data consistency.
- **Applying appropriate synchronization mechanisms:** using locks to provide exclusive access to shared resources, preventing concurrent modifications.
- **Implementing transaction management:** Use transactional mechanisms to ensure atomicity, consistency, isolation, and durability of data operations, thereby maintaining data integrity.

By implementing these measures, the voting system's global software control can effectively manage request initiation, workflow coordination, and synchronization, ensuring a reliable and efficient voting system.

3.5.8 Boundary Condition

Our project emphasizes the system's stability and reliability by accounting for its behavior in various boundary conditions. This section provides an overview of how the system handles startup, shutdown, errors, high user traffic, and exceptional situations.

The system will undergo a series of initialization processes during the startup phase. It will check dependencies, establish connections with databases or external services, and perform necessary configurations. To keep users informed, we plan to incorporate a startup progress indicator displaying initialization progress. Once the startup process is complete, the system will transition to an operational state, ready to handle user requests effectively.

When shutting down the system, we will implement a graceful shutdown procedure. This ensures that all ongoing operations are properly completed or safely terminated. For instance, we will provide any pending transactions that are finalized and unsaved user data saved securely. Following a well-defined shutdown process can bring the system offline without any data loss or corruption.

Since error conditions are inevitable, we will incorporate robust mechanisms to detect errors promptly. The system will log relevant information for analysis, allowing us to identify and resolve issues efficiently. Users will receive clear and informative error messages, helping them understand and troubleshoot problems. Depending on the severity of the error, the system will attempt automatic recovery or notify system administrators for further investigation and resolution. We aim to minimize disruptions, maintain data integrity, and restore regular system operation quickly.

To handle high user traffic, we will implement strategies to ensure optimal performance and responsiveness. Load balancing techniques will be utilized to distribute incoming requests across multiple servers or instances, preventing any single point of failure. Additionally, we will design the system to scale horizontally by adding more computing resources to handle increased demand effectively. Caching mechanisms, query optimization, and resource allocation algorithms will be implemented to manage high-user traffic efficiently, ensuring system stability and responsiveness even during peak usage periods.

We will incorporate fault tolerance and disaster recovery mechanisms to anticipate unusual scenarios such as network outages, hardware failures, or security breaches. Redundant servers

or backup systems will be deployed to ensure continuous availability. Regular data backups will be performed to safeguard data integrity, and stringent security measures will be implemented to prevent unauthorized access or data breaches. By proactively addressing these scenarios, we aim to minimize downtime and maintain the system's integrity and security.

If the voting system is blockchain-based, additional considerations will be considered. During the startup phase, the system will establish connections with the blockchain network, ensuring synchronization with distributed nodes and verifying the integrity of the blockchain ledger. The shutdown procedure will involve properly finalizing and committing any pending transactions to the blockchain, ensuring the immutability and transparency of the voting data. In addition, error-handling mechanisms will include smart contracts within the blockchain network to detect and handle exceptions or invalid transactions. The decentralized nature of the blockchain will contribute to fault tolerance, as redundant nodes and data replication across the network provide resilience against network outages and hardware failures.

Conclusion

Completing our documentation is crucial as we wrap up this project phase. We have documented our project's scope and limitations through requirement analysis and planning. By listing the functional requirements, we clearly understand what our project needs. Following these requirements, we have developed a medium feasibility estimate for the project. Additionally, creating diagrams has helped clarify the final system's appearance.

We have proposed a system that allows companies, entertainment awards, and election officials to create their votes using an easy and intuitive interface. The system handles much of the work and simplifies managing vote announcements.

As we move into the next phase of our project, our primary focus will be on developing the system. We will implement the features outlined in our documentation in the development phase, ensuring the system meets all specified requirements. We will also conduct thorough testing to verify that the system functions correctly and efficiently. Additionally, we will integrate user feedback to refine and improve the system further. By staying focused and adhering to our detailed plan, we aim to create a robust and user-friendly voting system that fulfills the needs of our users.

References

1. A. Hayes, "Blockchain Facts: What is it, how it works, and how it can be used," *Investopedia*, May 18, 2024. <https://www.investopedia.com/terms/b/blockchain.asp>
2. "Voting technology | MIT Election Lab." <https://electionlab.mit.edu/research/voting-technology>
3. M. H. Berenjestanaki, H. R. Barzegar, N. E. Ioini, and C. Pahl, "Blockchain-Based E-Voting Systems: A Technology review," *Electronics*, vol. 13, no. 1, p. 17, Dec. 2023, doi: 10.3390/electronics13010017.
4. GeeksforGeeks, "Django Project MVT Structure," *GeeksforGeeks*, Aug. 16, 2021. <https://www.geeksforgeeks.org/django-project-mvt-structure/>
5. R. Sheldon, "model-view-controller (MVC)," *WhatIs*, Sep. 12, 2023. <https://www.techtarget.com/whatis/definition/model-view-controller-MVC>
6. "Voting technology | MIT Election Lab." <https://electionlab.mit.edu/research/voting-technology>
7. Dennis, A., Wixom, B. H., & Roth, R. M. (2014). *Systems analysis and design*. John Wiley & Sons.
8. B. Bruegge and A. H. Dutoit, *Object-Oriented software engineering using UML, patterns, and Java*. 2009. [Online]. Available: http://library.upnvj.ac.id/index.php?p=show_detail&id=4080&keywords
9. GeeksforGeeks, "Activity Diagrams Unified Modeling Language (UML)," *GeeksforGeeks*, Jan. 15, 2024. <https://www.geeksforgeeks.org/unified-modeling-language-uml-activity-diagrams/>
10. B. Bruegge and A. H. Dutoit, *Object-Oriented software engineering using UML, patterns, and Java*. 2009. [Online]. Available: http://library.upnvj.ac.id/index.php?p=show_detail&id=4080&keywords
11. B. Bruegge and A. H. Dutoit, *Object-Oriented software engineering using UML, patterns, and Java*. 2009. [Online]. Available: http://library.upnvj.ac.id/index.php?p=show_detail&id=4080&keywords
12. R. Pressman and M. G. H. India, *Software Engineering: A Practitioner's Approach, 7Th Edition*. 2009. [Online]. Available: https://openlibrary.org/books/OL30677054M/Software_Engineering

13. I. Sommerville, *Engineering Software Products: An Introduction to modern software engineering*. 2019. [Online]. Available:
<https://dialnet.unirioja.es/servlet/libro?codigo=822265>
14. A. Dennis, B. H. Wixom, and D. Tegarden, *Systems Analysis and Design: An Object-Oriented Approach with UML*. 2016. [Online]. Available:
<http://ci.nii.ac.jp/ncid/BB18715383>
15. “Rational Software Architect 9.7.0,” Sep. 21, 2023.
<https://www.ibm.com/docs/en/rational-soft-arch/9.7.0?topic=diagrams-deployment>

Appendix

Appendix A: questionnaire for voter

Voting System Improvement Questionnaire

We value your feedback and aim to enhance the voting experience for all participants. Please take a few moments to complete this questionnaire to help us understand the challenges you have faced and gather valuable insights for improving the voting system. Your responses will remain anonymous and will be used solely for the purpose of system enhancement. Thank you for your valuable input!

1. Which types of voting processes have you participated in? Please select **all** that apply:

Check all that apply.

- ☐ Elections
- ☐ Surveys
- ☐ Option polls
- ☐ Awards or competitions
- ☐ Organizational decisions (e.g., board elections, policy choices)

2. On a scale of 1-4, how would you rate your **overall** experience with the existing voting systems?

Mark only one oval.

- ☐ 1 (Highly unsatisfactory)
- ☐ 2
- ☐ 3
- ☐ 4 (Highly satisfactory)

3. What are the major challenges or issues you have encountered while participating in the voting process? Please select all that apply:

Check all that apply.

- ☐ Long waiting times
- ☐ Lack of clear instructions
- ☐ Difficulty in understanding the ballot
- ☐ Limited voting options
- ☐ Inefficient result tabulation
- ☐ Security concerns

4. How important do you think it is to improve the existing voting systems?

Mark only one oval.

- ☐ Very important
- ☐ Somewhat important
- ☐ Neutral
- ☐ Not very important
- ☐ Not important at all

5. How would you rate the level of transparency in the current voting process?

Mark only one oval.

- ☐ Very transparent
- ☐ Somewhat transparent
- ☐ Neutral
- ☐ Somewhat opaque
- ☐ Very opaque

6. Which specific areas of the voting process do you believe need the most improvement? Please select all that apply:

Check all that apply.

- ☐ Registration process
- ☐ Ballot design and clarity
- ☐ Voting accessibility (physical or digital)
- ☐ Result tabulation and reporting
- ☐ Security measures

7. If you could introduce one change to enhance the voting experience, what would it be? Please select one option:

Mark only one oval.

- ☐ Improved voter education and awareness programs
- ☐ Enhanced accessibility options for voters with disabilities
- ☐ Streamlined and user-friendly online voting platform
- ☐ Implementing advanced security measures, such as biometric authentication

8. Are there any specific features or functionalities you would like to see in an improved voting system? Please select all that apply:

Check all that apply.

- ☐ Real-time result updates
- ☐ Option to verify individual vote status or receipt
- ☐ User-friendly interface with clear instructions
- ☐ Multiple language support

9. Any additional feedback or suggestions to improve the voting system?

This content is neither created nor endorsed by Google.

Google Forms